

TRƯỜNG ĐẠI HỌC

KHOA CÔNG NGHỆ THÔNG TIN

LUẬN VĂN TỐT NGHIỆP ĐẠI HỌC

Tối ưu ngân sách lời gọi mô hình ngôn ngữ lớn cho thiết kế heuristic đa mục tiêu dựa trên MPaGE

Dự án: mpage_bmab

Ngành: Khoa học máy tính

Hướng nghiên cứu: Generative AI và tối ưu tổ hợp đa mục tiêu

Công cụ biên soạn: XeLaTeX

Ngày 13 tháng 6 năm 2026

Tóm tắt

Dự án `mpage_bmab` nghiên cứu cách sử dụng ngân sách lời gọi mô hình ngôn ngữ lớn một cách hiệu quả hơn trong bài toán tự động thiết kế heuristic cho tối ưu tổ hợp đa mục tiêu. Nền tảng của dự án là MPaGE, một phương pháp kết hợp mô hình ngôn ngữ lớn, tiến hóa đơn điệu đa mục tiêu, lưới Pareto và phân cụm ngữ nghĩa để sinh các hàm heuristic. Phần mở rộng của dự án thay thế lịch lựa chọn cố định bằng một cơ chế bandit hai tầng có ràng buộc ngân sách: tầng thứ nhất chọn toán tử sinh heuristic, tầng thứ hai chọn cụm ngữ nghĩa trong từng thế hệ. Dự án cũng bổ sung theo dõi ngân sách, phát hiện trôi phân phối bằng Page–Hinkley, hàm thưởng có chặn, căn chỉnh tín hiệu thưởng với HV cuối của quần thể được quản lý và chỉ số diện tích dưới đường cong ngân sách AUBC.

Luận văn này tổng hợp cơ sở lý thuyết, kiến trúc hệ thống, chi tiết triển khai và phân tích thực nghiệm từ mã nguồn và tài liệu trong kho dự án. Bộ kết quả hiện tại gồm 160 lượt chạy trên bốn bài toán, bốn mức ngân sách và năm seed, so sánh cấu hình đầy đủ của `mpage_bmab` với biến thể `mpage_orig` được bọc lại để dùng cùng ngân sách. Kết quả cho thấy `mpage_bmab` cải thiện AUBC trung bình trong 15/16 tổ hợp bài toán–ngân sách và thắng 71/80 cặp chạy theo seed; tuy nhiên HV cuối cùng chỉ cải thiện trong 10/16 tổ hợp, do đó đóng góp chính của bộ kết quả này nên được hiểu là hiệu quả sử dụng ngân sách trong quá trình tìm kiếm, không phải ưu thế tuyệt đối ở trạng thái cuối. Với năm seed, kiểm định Wilcoxon hai phía không đạt mức ý nghĩa 0.05, nên các kết luận được trình bày theo hướng bằng chứng thực nghiệm và ý nghĩa thực tiễn thay vì khẳng định ý nghĩa thống kê mạnh.

Mục lục

1	Giới thiệu	1
1.1	Bối cảnh nghiên cứu	1
1.2	Đặt vấn đề	1
1.3	Động lực nghiên cứu	2
1.4	Mục tiêu nghiên cứu	2
1.5	Phạm vi nghiên cứu	3
1.6	Phương pháp nghiên cứu	4
1.7	Cấu trúc luận văn	4
2	Tổng quan và cơ sở lý thuyết	5
2.1	Tối ưu tổ hợp đa mục tiêu	5
2.2	Các hướng tiếp cận liên quan	5
2.3	Hypervolume và đánh giá Pareto front	6
2.4	MPaGE	6
2.5	Thiết kế heuristic bằng mô hình ngôn ngữ lớn	7
2.6	Multi-armed bandit và UCB	8
2.7	Budgeted bandit	8
2.8	Page–Hinkley và trôi phân phối	8
2.9	AUBC: diện tích dưới đường cong ngân sách	9
2.10	Kiểm kê kỹ thuật trong dự án	9
2.11	Nhận xét về khoảng trống của phương pháp hiện có	10
3	Proposed Method	11
3.1	Research Motivation and Design Rationale	11
3.2	Problem Formulation	12
3.2.1	Inner Multi-Objective Combinatorial Optimization Problem	12
3.2.2	Outer Heuristic-Design Problem	13

3.3	System Overview	14
3.4	Components Inherited from MPaGE	15
3.4.1	Population Representation and PFG Selection	15
3.4.2	LLM-Based Semantic Clustering	16
3.4.3	Mutation and Crossover Prompt Families	17
3.5	Hard Budget Model	17
3.6	Two-Level Budgeted Bandit Scheduler	18
3.6.1	Why a Two-Level Scheduler is Needed	18
3.6.2	Operator-Level UCB	18
3.6.3	Cluster-Level Budgeted UCB	18
3.7	Front-Aware Cluster Priors	19
3.8	Reward and Credit Assignment	20
3.8.1	Reward Components	20
3.8.2	Quality Signal	21
3.8.3	Diversity and Rank Smoothing	21
3.9	Non-Stationarity and Page–Hinkley Reset	22
3.10	Parent Selection and Variation Workflow	22
3.11	Complete Algorithm	23
3.12	Baseline and Ablation Design	25
4	Triển khai hệ thống	27
4.1	Cấu trúc mã nguồn	27
4.2	Môi trường phần mềm	27
4.3	Chuẩn bị bài toán đánh giá	28
4.4	Pipeline sinh và đánh giá heuristic	28
4.5	Cấu hình thực nghiệm	28
4.6	Script chạy, tổng hợp và so sánh	29
4.7	Artifact đầu ra	29
4.8	Tái lập kết quả	30
4.9	Các điểm cần thận trọng trong triển khai	30
5	Thực nghiệm và đánh giá	31
5.1	Câu hỏi thực nghiệm	31
5.2	Thiết lập thực nghiệm	31
5.3	Metric đánh giá	32

5.4	Kết quả AUBC	32
5.5	Đường cong ngân sách	34
5.6	HV cuối cùng	35
5.7	Tỷ lệ invalid/null-score và valid heuristic yield	38
5.8	Phân tích thống kê	40
5.9	Diễn giải theo từng nhóm bài toán	41
5.10	Hạn chế của thực nghiệm	42
5.11	Kết luận thực nghiệm	42
6	Kết luận và hướng phát triển	43
6.1	Tổng kết	43
6.2	Đóng góp chính	43
6.3	Phát hiện chính	44
6.4	Hạn chế	44
6.5	Hướng phát triển	45
6.6	Kết luận chung	45

Danh sách hình vẽ

2.1	Tổng quan MPaGE. Nguồn hình: figure/Overview.png.	7
2.2	Cơ chế chọn lựa dựa trên Pareto Front Grid của MPaGE. Nguồn hình: figure/Selection.png.	7
3.1	Logical architecture of BMAB-LLM. The method preserves MPaGE’s prompt, evaluation, semantic clustering, and population-management components, while adding budget accounting and adaptive bandit control. .	14
5.1	AUBC trung bình theo ngân sách. Nguồn: mpage_bmab/experiments/results/images/aubc_mean_by_budget.png.	33
5.2	Heatmap phần trăm thay đổi AUBC của BMAB so với MPaGE-orig. Nguồn: mpage_bmab/experiments/results/images/aubc_delta_percent_heatmap.png.	34
5.3	Đường cong HV theo ngân sách chuẩn hóa. Nguồn: các tệp budget_curves_B*.png trong mpage_bmab/experiments/results/images/. . .	35
5.4	HV cuối cùng trung bình theo ngân sách. Nguồn: mpage_bmab/experiments/results/images/hv_final_mean_by_budget.png.	37
5.5	Heatmap phần trăm thay đổi HV cuối cùng của BMAB so với MPaGE-orig. Nguồn: mpage_bmab/experiments/results/images/hv_final_delta_percent_heatmap.png.	37
5.6	Valid heuristic yield theo ngân sách. Nguồn: mpage_bmab/experiments/results/images/valid_yield_by_budget.png.	39
5.7	Heatmap phần trăm thay đổi valid heuristic yield. Nguồn: mpage_bmab/experiments/results/images/valid_yield_delta_percent_heatmap.png.	39
5.8	Tỷ lệ invalid/null-score theo ngân sách. Nguồn: mpage_bmab/experiments/results/images/invalid_null_rate_by_budget.png.	40
5.9	Phân phối chênh lệch AUBC theo cặp seed. Nguồn: mpage_bmab/experiments/results/images/paired_aubc_delta_boxplot.png.	41

Danh sách bảng

2.1	Kiểm kê kỹ thuật Generative AI và học máy xuất hiện trong dự án.	9
3.1	Main methodological modules in the implementation.	15
3.2	Default budget costs in the current implementation.	17
3.3	Main method variants relevant to the proposed method.	26
5.1	So sánh AUBC trung bình	32
5.2	So sánh HV cuối cùng trung bình	36
5.3	Số heuristic hợp lệ trên 100 lời gọi	38

Chương 1

Giới thiệu

1.1 Bối cảnh nghiên cứu

Tối ưu tổ hợp đa mục tiêu là một lớp bài toán trung tâm trong khoa học máy tính, xuất hiện trong định tuyến, lập lịch, lựa chọn tài nguyên, logistics và thiết kế hệ thống. Điểm khó của lớp bài toán này không chỉ nằm ở kích thước không gian nghiệm rời rạc, mà còn ở việc nhiều mục tiêu có thể mâu thuẫn nhau. Thay vì tìm một nghiệm duy nhất, thuật toán cần xấp xỉ một tập nghiệm Pareto, trong đó không nghiệm nào bị nghiệm khác trội hoàn toàn trên tất cả mục tiêu.

Trong các bài toán tối ưu tổ hợp đa mục tiêu, chất lượng của một heuristic phụ thuộc mạnh vào cấu trúc bài toán. Một heuristic tốt cho bài toán người du lịch hai mục tiêu chưa chắc hiệu quả cho bài toán định tuyến xe có ràng buộc tải trọng hoặc bài toán ba lô hai mục tiêu. Vì vậy, hướng tự động thiết kế heuristic bằng mô hình ngôn ngữ lớn đang trở thành một hướng nghiên cứu đáng chú ý: thay vì viết tay toàn bộ luật chọn lân cận, hệ thống dùng LLM để sinh, biến đổi và kết hợp các chương trình heuristic, sau đó đánh giá chúng bằng bộ mô phỏng tối ưu.

MPaGE đề xuất kết hợp mô hình ngôn ngữ lớn, tiến hóa đơn điệu đa mục tiêu, lưới Pareto và phân cụm ngữ nghĩa để thiết kế heuristic cho tối ưu tổ hợp đa mục tiêu [1]. Dự án `mpage_bmab` kế thừa nền tảng này và tập trung vào một câu hỏi hẹp hơn: khi số lời gọi LLM là tài nguyên hữu hạn, hệ thống nên phân bổ ngân sách sinh heuristic như thế nào để thu được chất lượng tốt sớm hơn?

1.2 Đặt vấn đề

Theo `mpage_bmab/documents/IDEA.md` và `mpage_bmab/documents/DOCUMENTATION.md`, điểm yếu được nhận diện trong MPaGE không nằm ở ý tưởng dùng LLM để sinh heuristic, mà nằm ở cơ chế phân bổ tài nguyên. MPaGE dùng lịch toán tử tương đối cố định

và lựa chọn cụm ngữ nghĩa theo cách đều hoặc ngẫu nhiên. Cách làm này đơn giản và phù hợp với một vòng lặp tiến hóa cơ sở, nhưng chưa trực tiếp trả lời câu hỏi: trong một ngân sách B lời gọi LLM, lời gọi tiếp theo nên dùng để đột biến heuristic nào, lai ghép cụm nào, hay khai thác hướng tìm kiếm nào?

Dự án `mpage_bmab` đặt bài toán ở mức outer-loop của thiết kế heuristic. Mỗi lời gọi LLM tạo ra một ứng viên chương trình. Ứng viên được biên dịch, đánh giá an toàn và gán một vector điểm số phản ánh chất lượng nghiệm và thời gian chạy. Hệ thống phải quyết định chuỗi hành động sinh heuristic sao cho đường cong chất lượng theo ngân sách tăng nhanh, đồng thời không phá vỡ các thành phần đã chứng minh hữu ích của MPaGE như PFG, phân cụm ngữ nghĩa và quần thể Pareto.

1.3 Động lực nghiên cứu

Động lực của dự án có thể tóm tắt thành ba quan sát có bằng chứng trực tiếp trong kho mã nguồn.

Thứ nhất, lời gọi LLM là chi phí thật. `mpage_bmab/budget.py` định nghĩa `BudgetTracker` và ép mỗi lượt sinh hoặc phân cụm phải được ghi nhận bằng `charge`. `mpage_bmab/main.py` và `mpage_bmab/mpage_orig.py` đều truyền tham số `--budget` để đảm bảo so sánh trong cùng số lời gọi.

Thứ hai, không phải mọi lời gọi LLM đều tạo ra heuristic hợp lệ. `llm4ad/base/evaluate.py` đánh giá chương trình sinh ra trong một tiến trình riêng và có thể trả về `None` khi mã không hợp lệ, timeout hoặc lỗi thực thi. Các tệp `mpage_bmab/experiments/results/validity_stats.csv` và `mpage_bmab/experiments/results/validity_cell_stats.csv` cho thấy tỷ lệ hợp lệ là một hiện tượng thực nghiệm cần theo dõi.

Thứ ba, kết quả cuối cùng không phản ánh đầy đủ hiệu quả ngân sách. `mpage_bmab/documents/METRICS_AUBC_HV.md` định nghĩa AUBC như diện tích dưới đường cong hypervolume theo ngân sách. Một phương pháp có thể đạt HV cuối tương đương nhưng đạt chất lượng tốt sớm hơn, khi đó AUBC sẽ cao hơn và có ý nghĩa trong bối cảnh chi phí LLM bị giới hạn.

1.4 Mục tiêu nghiên cứu

Luận văn này có các mục tiêu sau:

1. Phân tích nền tảng MPaGE và xác định các thành phần được giữ lại trong `mpage_bmab`. Nguồn chính gồm `README.md`, `llm4ad/method/LLMPFG/` và `mpage_bmab/documents/RETAINED_FROM_MPAGE.md`.

2. Mô tả phương pháp BMAB-LLM, bao gồm ngân sách cứng cho lời gọi LLM, bandit chọn toán tử, bandit chọn cụm, Page–Hinkley, hàm thưởng có nhiều chế độ, reward hướng HV cuối cùng, annealing thăm dò theo ngân sách và xả quần thể chờ trước khi ghi kết quả cuối. Nguồn chính gồm `mpage_bmab/bandit.py`, `mpage_bmab/reward.py`, `mpage_bmab/cluster_manager.py` và `mpage_bmab/bmab_lm.py`.
3. Phân tích triển khai hệ thống, pipeline đánh giá và cách tái lập thí nghiệm. Nguồn chính gồm `mpage_bmab/main.py`, `mpage_bmab/mpage_orig.py`, `mpage_bmab/experiments/run.py` và `mpage_bmab/experiments/aggregate.py`.
4. Tổng hợp kết quả thực nghiệm hiện có trong `mpage_bmab/experiments/results/`, bao gồm AUBC, HV cuối cùng, tỷ lệ invalid/null-score và valid heuristic yield.
5. Rút ra kết luận có kiểm soát: chỉ khẳng định những gì được hỗ trợ bởi mã nguồn, tài liệu và kết quả thực nghiệm; các thông tin thiếu sẽ được ghi rõ bằng câu “Không tìm thấy thông tin trong mã nguồn hoặc tài liệu của dự án.”

1.5 Phạm vi nghiên cứu

Phạm vi thực nghiệm của luận văn là bộ kết quả hiện có trong `mpage_bmab/experiments/results/`. Bộ này gồm hai phương pháp: `full`, tương ứng cấu hình đầy đủ của `mpage_bmab`, và `mpage_orig`, tương ứng MPaGE gốc được bọc lại để ghi nhận ngân sách theo cùng định dạng. Bốn bài toán được đánh giá là Bi-TSP, Tri-TSP, Bi-CVRP và Bi-KP. Bốn ngân sách lời gọi LLM là 25, 50, 100 và 200. Mỗi tổ hợp được chạy với năm seed từ 2025 đến 2029, theo cấu hình trong `mpage_bmab/experiments/configs.py`.

Các biến thể như `dense_reward`, `hybrid_reward`, `no_budget_anneal`, `no_ph`, `no_diversity`, `op_only` và `cluster_only` được xem là ablation của phương pháp chính. Những biến thể này hữu ích để kiểm tra riêng vai trò của tín hiệu reward, annealing theo ngân sách, Page–Hinkley, diversity reward và hai tầng bandit, nhưng không phải mọi ablation đều có kết quả đầy đủ trong thư mục thực nghiệm hiện có.

Luận văn không đánh giá lại các bảng kết quả lớn trong bài báo MPaGE gốc, vì kho hiện tại chỉ cung cấp kết quả thực nghiệm mới dưới dạng outer-loop heuristic-population HV/AUBC. Luận văn cũng không tuyên bố có huấn luyện LLM, fine-tuning, LoRA, RAG, diffusion model hay GAN, vì không tìm thấy thông tin trong mã nguồn hoặc tài liệu của dự án.

1.6 Phương pháp nghiên cứu

Phương pháp nghiên cứu của luận văn là phân tích thực chứng dựa trên mã nguồn và kết quả thực nghiệm. Trước hết, các tệp README, tài liệu thiết kế, mã nguồn MPaGE gốc, mã nguồn `mpage_bmab`, evaluator của từng bài toán và script thí nghiệm được đọc để xây dựng mô hình hệ thống. Sau đó, các tệp CSV và JSON trong `mpage_bmab/experiments/results/` được dùng để tổng hợp số liệu. Cuối cùng, các hình trong `mpage_bmab/experiments/results/images/` được đưa vào luận văn để kiểm tra trực quan các kết luận.

Về phân tích định lượng, luận văn ưu tiên AUBC vì đây là metric phù hợp với giả thiết ngân sách lời gọi LLM hữu hạn. HV cuối cùng được dùng như metric phụ để kiểm tra liệu hiệu quả sớm có đi kèm chất lượng cuối cùng hay không. Các thống kê thắng theo seed và kiểm định Wilcoxon được lấy từ `mpage_bmab/experiments/results/comparisons_aubc.csv` và `mpage_bmab/experiments/results/comparisons_hv_final.csv`.

1.7 Cấu trúc luận văn

Chương 2 trình bày tổng quan tối ưu đa mục tiêu, MPaGE, thiết kế heuristic bằng LLM, bandit và các metric đánh giá. Chương 3 mô tả phương pháp đề xuất BMAB-LLM, bao gồm mô hình hóa bài toán, kiến trúc và thuật toán. Chương 4 trình bày chi tiết triển khai, pipeline chạy thí nghiệm và các artifact được sinh ra. Chương 5 phân tích kết quả thực nghiệm hiện có. Chương 6 kết luận, nêu hạn chế và hướng phát triển.

Chương 2

Tổng quan và cơ sở lý thuyết

2.1 Tối ưu tổ hợp đa mục tiêu

Một bài toán tối ưu tổ hợp đa mục tiêu có thể được viết dưới dạng:

$$\min_{x \in \mathcal{X}} F(x) = (f_1(x), f_2(x), \dots, f_m(x)), \quad (2.1)$$

trong đó $\mathcal{X}$ là không gian nghiệm rời rạc và m là số mục tiêu. Với hai nghiệm x và y , ta nói x trội Pareto y nếu $f_i(x) \leq f_i(y)$ với mọi i và tồn tại ít nhất một mục tiêu j sao cho $f_j(x) < f_j(y)$. Tập các nghiệm không bị trội tạo thành xấp xỉ Pareto front.

Trong dự án này, các heuristic do LLM sinh ra không trực tiếp là nghiệm cuối cùng của bài toán TSP, CVRP hoặc KP. Thay vào đó, chúng là hàm chọn lân cận `select_neighbor`, được dùng bên trong một vòng tìm kiếm cục bộ hoặc tiến hóa đơn giản. Điều này được thể hiện trong các template bài toán như `llm4ad/task/optimization/bi_tsp/template.py`, `llm4ad/task/optimization/tri_tsp/template.py`, `llm4ad/task/optimization/bi_cvrp/template.py` và `llm4ad/task/optimization/bi_kp/template.py`. Vì vậy, bài toán outer-loop của MPaGE và `mpage_bmab` là tối ưu hóa chính các heuristic.

2.2 Các hướng tiếp cận liên quan

Trước khi LLM được dùng để sinh heuristic, tối ưu đa mục tiêu thường dựa trên các thuật toán tiến hóa hoặc tìm kiếm cục bộ. NSGA-II dùng non-dominated sorting và crowding distance để cân bằng hội tụ và đa dạng [2]. MOEA/D phân rã bài toán đa mục tiêu thành nhiều bài toán vô hướng con [3]. SEMO là một khung tiến hóa đơn điệu thường được dùng để phân tích và thiết kế thuật toán cho bài toán đa mục tiêu rời rạc [4]. MPaGE kế thừa tinh thần SEMO nhưng thay toán tử thủ công bằng toán tử do LLM sinh và chỉnh sửa.

Một nhánh khác là lựa chọn toán tử thích nghi trong thuật toán tiến hóa. Các phương pháp adaptive operator selection dùng tín hiệu phần thưởng để ưu tiên toán tử có đóng góp tốt hơn trong từng giai đoạn tìm kiếm [5]. `mpage_bmab` có quan hệ gần với hướng này, nhưng điểm khác là mỗi hành động không chỉ là một toán tử tiến hóa rỗng, mà là một lời gọi LLM có chi phí ngân sách rõ ràng.

Trong hướng Generative AI, các công trình như FunSearch cho thấy LLM có thể được dùng để sinh chương trình và khám phá lời giải toán học thông qua vòng lặp sinh–đánh giá [6]. MPaGE và `mpage_bmab` áp dụng tinh thần tương tự vào thiết kế heuristic cho tối ưu tổ hợp đa mục tiêu: chương trình do LLM sinh không được tin ngay, mà phải được thực thi và đánh giá định lượng.

2.3 Hypervolume và đánh giá Pareto front

Hypervolume là chỉ số phổ biến để đo chất lượng một tập nghiệm không bị trội. Với một điểm tham chiếu r , hypervolume đo thể tích vùng trong không gian mục tiêu bị chi phối bởi tập nghiệm và bị chặn bởi r [7, 8]. Trong mã nguồn `mpage_bmab`, hàm `hypervolume` được cài đặt trong `mpage_bmab/reward.py`; nếu thư viện `pymoo` khả dụng, hệ thống dùng `pymoo.indicators.hv.HV` [9], nếu không sẽ dùng fallback cho trường hợp hai chiều.

Điểm cần phân biệt là dự án có hai cấp hypervolume. Cấp bên trong là HV của nghiệm được sinh bởi một heuristic trên một bài toán cụ thể, ví dụ Bi-TSP hoặc Bi-CVRP. Cấp bên ngoài là HV của quần thể heuristic, trong đó mỗi heuristic có điểm số dạng $(-HV_{\text{inner}}, \text{runtime})$. `mpage_bmab/documents/METRICS_AUBC_HV.md` nhấn mạnh rằng HV/AUBC trong kết quả hiện tại là metric ở cấp quần thể heuristic, không phải bằng HV nghiệm chuẩn hóa trong bài báo MPaGE gốc.

2.4 MPaGE

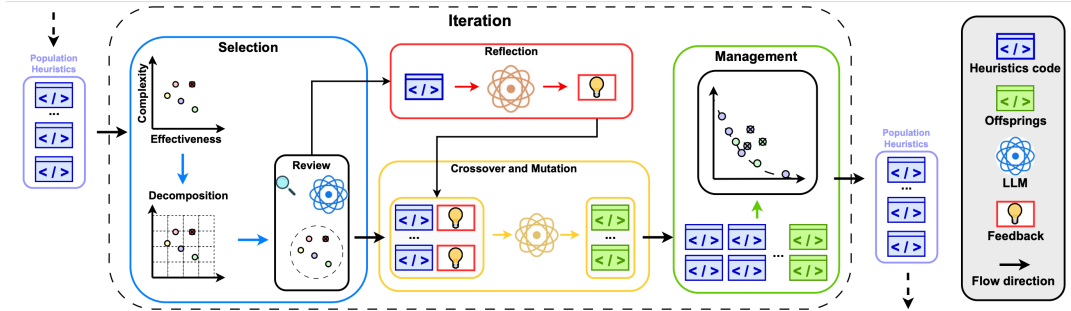
MPaGE là phương pháp thiết kế heuristic dùng LLM cho tối ưu tổ hợp đa mục tiêu. Về mặt phương pháp, MPaGE kết hợp ba ý tưởng chính.

Thứ nhất là vòng lặp tiến hóa đơn điệu đa mục tiêu. Các heuristic được giữ trong một quần thể, đánh giá bằng vector mục tiêu và chọn lọc bằng quan hệ không bị trội cùng crowding distance. Cài đặt nằm trong `llm4ad/method/LLMPFG/population.py`.

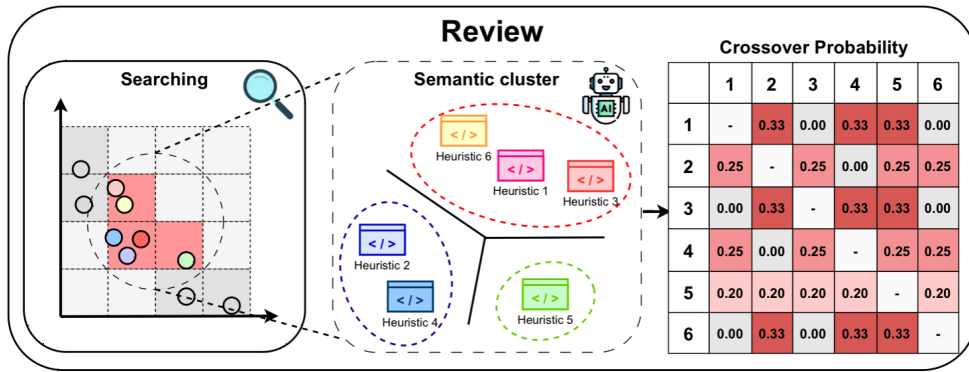
Thứ hai là Pareto Front Grid. PFG chia không gian mục tiêu thành các ô lưới để ưu tiên chọn heuristic ở những vùng đa dạng của Pareto front. Trong mã nguồn, các tham số như `pfg_gk`, `pfg_sigma` và `pfg_epsilon` xuất hiện trong `llm4ad/method/LLMPFG/population.py` và được giữ lại trong `mpage_bmab/documents/HYPERPARAMETER`

S.md. Hình 2.2 minh họa cơ chế chọn lựa được cung cấp trong thư mục figure/.

Thứ ba là phân cụm ngữ nghĩa bằng LLM. MPaGE yêu cầu LLM nhóm các heuristic dựa trên mô tả ý tưởng hoặc mã nguồn, sau đó dùng đột biến trong cụm và lai ghép giữa cụm để tăng đa dạng. Prompt phân cụm được định nghĩa trong `llm4ad/method/LLMPFG/prompt.py`. Hình 2.1 cho thấy tổng quan phương pháp MPaGE.



Hình 2.1: Tổng quan MPaGE. Nguồn hình: figure/Overview.png.



Hình 2.2: Cơ chế chọn lựa dựa trên Pareto Front Grid của MPaGE. Nguồn hình: figure/Selection.png.

2.5 Thiết kế heuristic bằng mô hình ngôn ngữ lớn

Trong dự án, LLM được dùng để sinh mã nguồn Python cho hàm heuristic, không dùng để trực tiếp giải bài toán tối ưu. Các prompt trong `llm4ad/method/LLMPFG/prompt.py` định nghĩa năm kiểu thao tác: khởi tạo I1, khai thác E1/E2 và đột biến M1/M2. `mpage_bmab/bmab_llm.py` dùng lại các prompt này thông qua sampler của MPaGE và chỉ thay đổi cách chọn thời điểm, toán tử và cụm.

Về mặt Generative AI, đây là bài toán program synthesis có kiểm chứng bằng thực thi. LLM sinh một chương trình ứng viên, sau đó `llm4ad/base/evaluate.py` chạy chương trình trong môi trường đánh giá an toàn. Cách làm này khác với fine-tuning hoặc học có giám sát truyền thống: không có dữ liệu nhãn để huấn luyện lại mô hình, mà chất lượng được cải

thiện qua vòng lặp tạo–đánh giá–chọn lọc. Trong kho mã nguồn không tìm thấy bằng chứng về huấn luyện Transformer, fine-tuning, LoRA, PEFT hoặc distillation.

2.6 Multi-armed bandit và UCB

Multi-armed bandit mô hình hóa bài toán chọn một hành động trong nhiều hành động khi phần thưởng chưa biết trước. Với UCB1, mỗi arm a được chọn theo điểm:

$$\text{UCB}(a) = \hat{\mu}_a + c\sqrt{\frac{\log t}{n_a}}, \quad (2.2)$$

trong đó $\hat{\mu}_a$ là phần thưởng trung bình, n_a là số lần arm đã được chọn và t là tổng số lượt chọn [10, 11]. Thành phần thứ nhất khai thác arm tốt đã biết, thành phần thứ hai khuyến khích khám phá arm ít được thử.

`mpage_bmab` dùng UCB ở hai tầng. `mpage_bmab/bandit.py` định nghĩa `OperatorBandit` để chọn giữa hai nhóm toán tử chính: `mutate` và `crossover`. Cùng tệp này định nghĩa `ClusterBandit` để chọn cặp cụm–toán tử trong từng thế hệ. Thiết kế này phản ánh nhận xét trong `mpage_bmab/documents/IDEA.md`: một toán tử tốt toàn cục chưa chắc tốt ở mọi cụm ngữ nghĩa, và một cụm tốt có thể trở nên kém hữu ích khi bị khai thác quá nhiều.

2.7 Budgeted bandit

Trong bandit có ràng buộc ngân sách, mỗi hành động tiêu tốn một chi phí và thuật toán phải tối đa hóa phần thưởng trước khi ngân sách cạn [12]. Với bài toán LLM, chi phí tự nhiên là số lời gọi mô hình. `mpage_bmab/budget.py` hiện thực hóa ý tưởng này bằng `BudgetTracker`. Mỗi lời gọi sinh hoặc phân cụm đều phải đi qua `charge`; khi ngân sách không đủ, hành động bị từ chối.

Trong `mpage_bmab/bandit.py`, `OperatorBandit` dùng phần thưởng trên chi phí, còn `ClusterBandit` bổ sung hệ số ngân sách còn lại. Cơ chế này không phải một lời giải tối ưu lý thuyết cho mọi biến thể budgeted MAB, nhưng phù hợp với yêu cầu kỹ thuật của dự án: đơn giản, ổn định, tích hợp được vào MPaGE và cho phép theo dõi quyết định theo từng lời gọi.

2.8 Page–Hinkley và trôi phân phối

Phần thưởng của một cụm hoặc toán tử không nhất thiết ổn định. Khi quần thể đã khai thác một vùng nào đó, cùng một cụm heuristic có thể giảm ích lợi biên. Page–Hinkley là một

kiểm định online để phát hiện thay đổi trung bình chuỗi quan sát [13]. Trong `mpage_bma`
`b/bandit.py`, trạng thái `PageHinkleyState` cập nhật trung bình online và tổng tích
lũy; khi độ lệch giữa cực đại lịch sử và tổng hiện tại vượt ngưỡng, arm được xem là bị trôi
và thông kê arm đó được reset.

Tài liệu `mpage_bma`/`documents/PAGE_HINKLEY.md` và `mpage_bma`/`docum`
`ents/HYPERPARAMETERS.md` ghi các tham số mặc định $\delta = 0.005$ và ngưỡng 0.5. Trong
cấu hình `full`, Page–Hinkley được bật; các ablation như `no_ph` có trong mã và tài liệu
nhưng không xuất hiện trong bộ kết quả hiện có.

2.9 AUBC: diện tích dưới đường cong ngân sách

AUBC được định nghĩa trong `mpage_bma`/`documents/METRICS_AUBC_HV.md` và cài
đặt trong `mpage_bma`/`profiler.py`. Với đường cong $HV(b)$ theo ngân sách tiêu thụ b ,
AUBC được tính xấp xỉ bằng quy tắc hình thang:

$$\text{AUBC} = \frac{1}{B} \int_0^B HV(b) db. \quad (2.3)$$

Profiler neo đường cong tại $(0, 0)$ và kéo phẳng giá trị cuối nếu quá trình kết thúc
trước B . Metric này có ý nghĩa khi chi phí sinh heuristic là yếu tố quan trọng: hai phương
pháp có thể có HV cuối tương tự nhau, nhưng phương pháp đạt HV cao sớm hơn sẽ có AUBC
lớn hơn.

2.10 Kiểm kê kỹ thuật trong dự án

Bảng 2.1 tổng hợp các kỹ thuật Generative AI và học máy được tìm thấy hoặc không tìm
thấy trong kho mã nguồn. Bảng này có vai trò kiểm soát phạm vi: luận văn chỉ phân tích các
kỹ thuật có bằng chứng.

Bảng 2.1: Kiểm kê kỹ thuật Generative AI và học máy xuất hiện trong dự án.

Kỹ thuật	Trạng thái	Bằng chứng trong kho mã nguồn
Mô hình ngôn ngữ lớn	Có	<code>README.md</code> , <code>mpage_bma</code> / <code>README.md</code> , <code>mpage_bma</code> / <code>main.py</code> , <code>mpage_bma</code> / <code>cluster_manager.py</code> .
Prompt engineering	Có	Các mẫu prompt I1, E1, E2, M1, M2 và prompt phân cụm trong <code>llm4ad/method/LMPFG/prompt.py</code> .

Kỹ thuật	Trạng thái	Bằng chứng trong kho mã nguồn
Bandit / học tăng cường nhẹ	Có	UCB1, Budgeted UCB và Page–Hinkley trong <code>mpage_bmab/bandit.py</code> .
Đánh giá tự động code sinh bởi LLM	Có	<code>llm4ad/base/evaluate.py</code> , các evaluator trong <code>llm4ad/task/optimization/</code> .
RAG	Không thấy	Không tìm thấy thông tin trong mã nguồn hoặc tài liệu của dự án.
Fine-tuning, PEFT, LoRA	Không thấy	Không tìm thấy thông tin trong mã nguồn hoặc tài liệu của dự án.
Diffusion model, GAN	Không thấy	Không tìm thấy thông tin trong mã nguồn hoặc tài liệu của dự án.
Knowledge distillation	Không thấy	Không tìm thấy thông tin trong mã nguồn hoặc tài liệu của dự án.
Multi-agent system	Không thấy	Hệ thống có vòng lặp LLM–tiến hóa và hai bandit, nhưng không tìm thấy bằng chứng về nhiều agent tự trị.

2.11 Nhận xét về khoảng trống của phương pháp hiện có

MPaGE đã giải quyết tốt bài toán tạo đa dạng heuristic bằng LLM và quần thể Pareto, nhưng lịch phân bổ lời gọi LLM còn ít thích nghi với tín hiệu thực nghiệm trong quá trình chạy. `mpage_bmab` bổ sung bandit để dùng chính kết quả đánh giá heuristic làm tín hiệu điều khiển. Tuy nhiên, do hệ thống vẫn phụ thuộc vào API LLM đóng và prompt sinh mã, nó còn chịu rủi ro về tính không quyết định, lỗi cú pháp, lỗi runtime và chi phí. Các rủi ro này được phản ánh trong thiết kế `BudgetTracker`, `SecureEvaluator`, `RewardComputer` và các thống kê `invalid/null-score`.

Chương 3

Proposed Method

3.1 Research Motivation and Design Rationale

The proposed method, denoted BMAB-LLM, is designed as a budget-aware extension of the original MPaGE framework. The central observation behind the project is that LLM-driven heuristic design is not only an optimization problem over generated programs, but also a resource-allocation problem. Each heuristic-generation, clustering, or optional reflection step consumes an LLM call, and therefore consumes time, money, and limited experimental budget. A method that produces high-quality heuristics only after many unproductive calls is less useful than a method that converts early calls into a strong heuristic Pareto front.

The original MPaGE paper formulates automatic heuristic design for multi-objective combinatorial optimization problems (MOCOPs) as a Language Multi-Criteria Heuristic Design problem. It combines three important ingredients: a SEMO-style heuristic population, Pareto-Front-Grid (PFG) elite selection, and LLM-based semantic clustering of heuristic code [1, 4]. This design is well motivated because it searches for a population of heuristics that trade off solution quality and runtime, rather than a single best heuristic. It also recognizes that code-level diversity is not equivalent to semantic diversity: two heuristics may be syntactically different but implement nearly identical search logic.

However, the baseline MPaGE workflow allocates generation effort in a mostly predetermined manner. Each generation repeatedly selects elites with PFG, groups promising heuristics into semantic clusters, applies mutation or crossover prompts, evaluates the generated program, and updates the population. This loop is effective for maintaining semantic and objective-space diversity, but it does not explicitly learn which operator or which semantic region is producing the most useful heuristics under the current budget. The stopping condition is also naturally expressed in terms of iterations or sample counts, whereas practical LLM-based search is constrained by a finite number of API calls. Under a strict budget B , the system should therefore learn where to spend the remaining

LLM calls instead of treating all operators and clusters as equally promising.

BMAB-LLM therefore keeps the productive heuristic-generation mechanisms of MPaGE and replaces only the scheduling layer. In particular, the project preserves the original task templates, prompt taxonomy, secure evaluator, PFG-based population management, and LLM semantic clustering implementation. The new contribution is a two-level budgeted multi-armed bandit controller that decides which variation operator and which semantic cluster should receive each LLM call. This design follows a conservative research principle: the comparison against MPaGE is made meaningful by reusing the same generation and evaluation substrate, while changing the decision policy that allocates budget.

3.2 Problem Formulation

3.2.1 Inner Multi-Objective Combinatorial Optimization Problem

Let $\mathcal{X}$ be a finite or discrete feasible set for a MOCOP, and let

$$f(x) = (f_1(x), f_2(x), \dots, f_M(x)), \quad x \in \mathcal{X}, \quad (3.1)$$

where all objectives are minimized. For two feasible solutions $x_a, x_b \in \mathcal{X}$, x_a Pareto-dominates x_b , written $x_a \prec x_b$, if

$$f_i(x_a) \leq f_i(x_b) \quad \forall i \in \{1, \dots, M\}, \quad \exists j : f_j(x_a) < f_j(x_b). \quad (3.2)$$

The Pareto front is the image of the non-dominated solution set in objective space. Its quality is commonly summarized by the hypervolume indicator $HV_r(\cdot)$ with respect to a reference point r [7, 8]. For a finite objective set Y in a minimization problem, the hypervolume is the Lebesgue measure of the region dominated by the non-dominated points in Y and bounded by r :

$$HV_r(Y) = \lambda \left(\bigcup_{y \in \text{ND}(Y)} [y_1, r_1] \times \dots \times [y_M, r_M] \right),$$

where $\text{ND}(Y)$ denotes the non-dominated subset and $\lambda(\cdot)$ denotes M -dimensional volume. A larger hypervolume means that the obtained Pareto approximation is both closer to the ideal region and more broadly spread across trade-offs. The benchmark evaluators in this project use this quantity as the inner solution-quality signal and execution time as the efficiency signal.

3.2.2 Outer Heuristic-Design Problem

Following the LMHD formulation in the original MPaGE paper, a heuristic $h \in \mathcal{H}$ is treated as a program that maps the current SEMO archive to a new candidate solution:

$$h : 2^{\mathcal{X}} \rightarrow \mathcal{X}, \quad x' = h(A), \quad (3.3)$$

where A is the current archive of non-dominated solutions maintained by the inner solver. The evaluator assigns each generated heuristic a vector-valued score

$$E(h) = (e_1(h), e_2(h)) = (-HV_{\text{inner}}(h), T(h)). \quad (3.4)$$

Equation (3.3) should be read as the interface between heuristic design and the inner solver. The LLM does not directly output a TSP tour, CVRP route, or knapsack solution. Instead, it outputs a heuristic program h . During evaluation, the inner SEMO-style solver repeatedly calls h with its current archive A and expects one candidate solution x' . The quality of h is therefore determined indirectly by the Pareto archive that the inner solver obtains after running with that heuristic.

Here $HV_{\text{inner}}(h)$ is the hypervolume achieved by the solutions produced when h is used inside the benchmark solver, and $T(h)$ is the measured runtime. More concretely, if the inner run using h returns a non-dominated solution archive A_h , then $HV_{\text{inner}}(h)$ is computed from the objective vectors $\{f(x) : x \in A_h\}$ using the task-specific reference point. This is the *inner hypervolume*: it measures how good one generated heuristic is at solving the underlying combinatorial optimization task.

The first coordinate in $E(h)$ is negated because the outer population-management code is written as a minimization procedure. Thus, a heuristic with a larger inner hypervolume receives a smaller first score coordinate. The second coordinate is runtime, so smaller is also better. This score convention is implemented by the benchmark evaluators in the vendored MPaGE task modules.

At the outer level, the algorithm maintains a population $\mathcal{P}_t \subset \mathcal{H}$ of valid heuristics and aims to approximate the non-dominated set in the heuristic score space. Let $F_t = \{E(h) : h \in \mathcal{P}_t\}$ be the outer heuristic front at step t . Given an LLM-call budget B , BMAB-LLM is optimized for two related criteria:

$$\max HV_{\text{outer}}(B) = \max HV_r(F_B), \quad (3.5)$$

$$\max \text{AUBC} = \max \frac{1}{B} \int_0^B HV_{\text{outer}}(b) db, \quad (3.6)$$

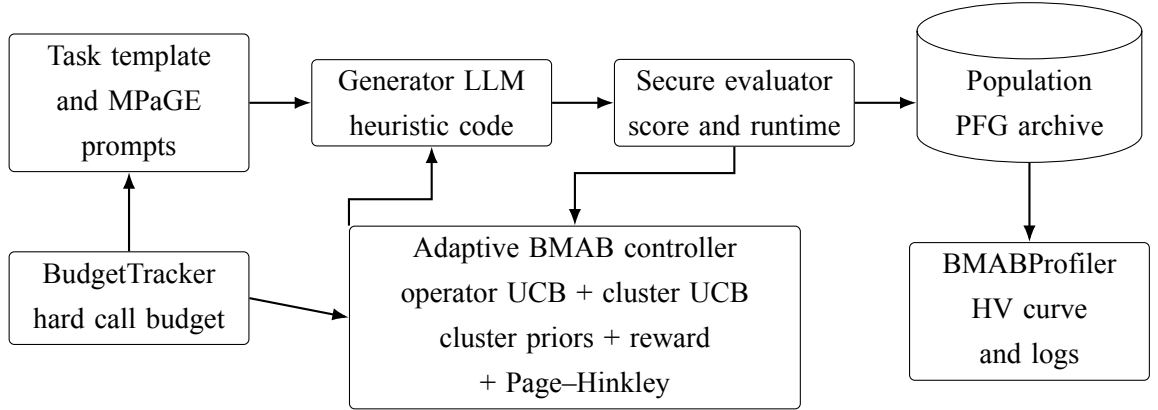
subject to

$$\sum_{t=1}^T c_t \leq B. \quad (3.7)$$

The quantity HV_{outer} is the *outer hypervolume*: it is computed over heuristic score vectors rather than over task solutions. A high outer hypervolume means that the final population contains useful trade-offs between inner solution quality and runtime, for example heuristics that achieve high inner HV quickly and heuristics that are slower but stronger. The final HV objective measures terminal population quality, while AUBC measures how rapidly the method converts budget into Pareto-front quality over the whole run. In the current experiments, the default cost model is call-based, i.e. one LLM request consumes one budget unit. The budget tracker also supports a token-labeled accounting mode, but the implemented main loop and reported experiments use fixed call costs.

3.3 System Overview

BMAB-LLM is organized as an adaptive orchestration layer around the MPaGE primitives. Figure 3.1 summarizes the data and control flow.



Hình 3.1: Logical architecture of BMAB-LLM. The method preserves MPaGE’s prompt, evaluation, semantic clustering, and population-management components, while adding budget accounting and adaptive bandit control.

The implementation is centered in `bmab_llm.py`. The main supporting modules are summarized in Table 3.1.

Bảng 3.1: Main methodological modules in the implementation.

Module	Methodological role
<code>bmab_llm.py</code>	Main orchestrator. It owns the evolutionary loop, calls PFG selection, requests semantic clusters, selects bandit arms, samples offspring, computes rewards, and flushes the final population.
<code>budget.py</code>	Hard budget tracker. It stores total, remaining, and consumed budget and records a history entry for every charged LLM call.
<code>bandit.py</code>	Two-level adaptive scheduler: persistent operator UCB, per-generation cluster UCB, and Page–Hinkley drift detection.
<code>cluster_manager.py</code>	Wrapper around MPaGE semantic clustering. It charges cluster calls, validates or repairs partitions, and computes cluster-quality priors.
<code>reward.py</code>	Scalar credit assignment for bandit learning, including HVI, final managed-population HV gain, diversity gain, rank smoothing, and invalid-code penalties.
<code>profiler.py</code>	Records budget-vs-HV curves, AUBC, budget history, bandit state, and population artifacts for analysis.

3.4 Components Inherited from MPaGE

BMAB-LLM intentionally retains the core generative and evaluative substrate of MPaGE. This inheritance is documented in the project design notes and is visible in the vendored MPaGE subtree. The retained components are important because they isolate the proposed contribution to budget-aware scheduling rather than changing the benchmark, prompt templates, or evaluation logic.

3.4.1 Population Representation and PFG Selection

Each individual is represented as a Python function together with a natural-language algorithm description, evaluation score, sample time, and execution time. The population class inherited from MPaGE maintains a managed set of heuristics using non-dominated sorting and diversity-preserving truncation.

PFG selection partitions the two-dimensional heuristic objective space into grid cells.

Let $P_t = \{E(h_i) : h_i \in \mathcal{P}_t\}$ be the current score set. For each objective $j \in \{1, 2\}$, define the ideal and nadir values

$$z_j^* = \min_{h \in \mathcal{P}_t} e_j(h), \quad z_j^n = \max_{h \in \mathcal{P}_t} e_j(h). \quad (3.8)$$

With K_j segments and a small margin σ , the grid width is approximately

$$\Delta_j = \frac{z_j^n - z_j^* + \sigma}{K_j}. \quad (3.9)$$

A heuristic is assigned to a grid index

$$g_j(h) = \left\lfloor \frac{e_j(h) - z_j^*}{\Delta_j} \right\rfloor, \quad G(h) = (g_1(h), g_2(h)). \quad (3.10)$$

Within each occupied cell, non-dominated representatives are retained. The union of retained representatives forms the elite set used for reproduction. This mechanism is inherited from MPaGE and is implemented in the vendored MPaGE population manager. In the current BMAB loop, the PFG-selected slice is used as a high-quality signal for parent weighting, while the wider managed population is made available to the clustering layer to avoid over-narrow clusters.

3.4.2 LLM-Based Semantic Clustering

The original MPaGE paper argues that objective-space or syntax-only diversity is insufficient for heuristic design. Two code snippets can have different syntax but the same algorithmic logic, and two heuristics with similar performance may use distinct search strategies. MPaGE addresses this by prompting an LLM to group elite heuristics according to semantic and behavioral similarity.

Formally, for a candidate set $P = \{h_1, \dots, h_n\}$, the clustering module returns

$$\text{SemClust}(P) = \{C_1, \dots, C_K\}, \quad (3.11)$$

such that

$$\bigcup_{k=1}^K C_k = P, \quad C_i \cap C_j = \emptyset \ (i \neq j). \quad (3.12)$$

The implementation calls the MPaGE cluster prompt through `cluster_manager.py`. If the LLM response is malformed, incomplete, or unavailable, the method falls back to singleton clusters. This design preserves semantic clustering when possible while ensuring that the evolutionary loop remains robust to LLM-format failures.

3.4.3 Mutation and Crossover Prompt Families

MPaGE defines a prompt taxonomy for initialization, mutation, and crossover. BMAB-LLM keeps this taxonomy. Initialization uses the I1 prompt to create zero-shot heuristics. Mutation uses M1 or M2 to modify a single parent from the selected cluster. Crossover uses E1 or E2 to combine one parent from the selected cluster with a second parent outside that cluster. Thus, BMAB preserves MPaGE’s core semantic rule:

$$\text{mutation} = \text{intra-cluster local refinement}, \quad \text{crossover} = \text{inter-cluster recombination}. \quad (3.13)$$

The difference is not the prompt content but the policy that decides when a mutation or crossover prompt should be used, and which cluster should provide the parent.

3.5 Hard Budget Model

The LLM-call budget is treated as a first-class constraint. Let B be the total budget and b_t be the remaining budget before action t . Each LLM action has cost c_t , and the transition is

$$b_{t+1} = b_t - c_t, \quad b_t \geq c_t. \quad (3.14)$$

If the system cannot afford an action, the action is not issued. The current implementation charges budget before the heuristic-generation call or cluster call is sent. This is important experimentally: invalid code, parsing failure, timeout, and non-improving offspring are still charged because the API resource has already been consumed.

In the default call-based mode, the following costs are used:

Bảng 3.2: Default budget costs in the current implementation.

Action	Cost
Initialization call (I1)	1
Heuristic-generation call (M1/M2/E1/E2)	1
Semantic clustering call	1
Optional review/suggestion call	1

The budget state is recorded through `BudgetTracker.history`, and the profiler later writes `budget_history.json`. This gives the experimental harness a direct account of how the budget was spent.

3.6 Two-Level Budgeted Bandit Scheduler

3.6.1 Why a Two-Level Scheduler is Needed

A direct bandit over individual heuristics is not viable because the heuristic space is open-ended: every LLM call can create a previously unseen program. A direct persistent bandit over semantic cluster IDs is also problematic because clusters are reconstructed at each generation and their numeric labels are not stable across generations. BMAB-LLM therefore separates the adaptive decision into two levels:

1. a persistent operator-level bandit over the stable action set $\mathcal{O} = \{\mu, \chi\}$, where μ denotes mutation and χ denotes crossover;
2. a per-generation cluster-level bandit over arms (k, o) , where k indexes one semantic cluster in the current generation and $o \in \mathcal{O}$ is the selected operator.

This decomposition lets the method learn long-term operator utility while still adapting to the short-lived semantic cluster structure of the current population.

3.6.2 Operator-Level UCB

For each operator $o \in \mathcal{O}$, the operator bandit stores the number of pulls n_o , cumulative reward S_o^R , and cumulative cost S_o^c . Its empirical utility is reward per cost:

$$\hat{\rho}_o = \frac{S_o^R}{\max(S_o^c, \varepsilon)}. \quad (3.15)$$

The selected operator maximizes a UCB score:

$$U_{\text{op}}(o) = \hat{\rho}_o + c_{\text{op}} \sqrt{\frac{2 \log N_{\text{op}}}{n_o}}, \quad (3.16)$$

where $N_{\text{op}} = \sum_{o' \in \mathcal{O}} n_{o'}$ and c_{op} controls exploration. The implementation initializes each operator with an optimistic prior count and prior reward, preventing division by zero and encouraging early exploration. This is implemented in `bandit.py` by the `OperatorBandit` class.

3.6.3 Cluster-Level Budgeted UCB

At the beginning of each generation, `ClusterManager` returns a partition $\{C_1, \dots, C_K\}$ and a quality prior q_k for each cluster. At this point, q_k can be understood as a scalar estimate of how promising cluster C_k is before any new offspring are generated in the current

generation; its formal front-aware definition is given in Section 3.7. The cluster bandit then creates arms (k, o) for all clusters and operator labels. Each arm stores cumulative reward and cumulative cost, and its exploitation term is

$$\hat{\rho}_{k,o} = \frac{S_{k,o}^R}{\max(S_{k,o}^c, \varepsilon)}. \quad (3.17)$$

The cluster-level UCB score is

$$U_{\text{cl}}(k, o) = \hat{\rho}_{k,o} + c_{\text{cl}} \alpha_t \sqrt{\frac{2 \log N_{\text{cl}}}{n_{k,o}}}, \quad (3.18)$$

where

$$\alpha_t = \left(\frac{b_t}{B} \right)^{\gamma_b} \quad (3.19)$$

when budget annealing is enabled, and $\alpha_t = 1$ otherwise. The factor α_t decreases as the budget is exhausted, making the controller more exploitative near the end of the run. This directly addresses the finite-horizon nature of the task: late calls have little time to recover from risky exploration.

The cluster bandit is warm-started from both cluster quality and operator priors. Let p_o be the softmax-normalized operator utility from the persistent operator bandit. The initial reward assigned to arm (k, o) is proportional to

$$r_{k,o}^{(0)} = \frac{1}{2} (q_k + p_o). \quad (3.20)$$

This warm start transfers information from the persistent operator layer into the generation-local cluster layer without assuming that cluster IDs persist across generations.

3.7 Front-Aware Cluster Priors

The semantic cluster partition identifies behavioral groups, but it does not by itself rank those groups. The current implementation therefore computes a front-aware prior for each cluster. For cluster C_k , define $S_k = \{E(h) : h \in C_k\}$ and $S = \{E(h) : h \in \mathcal{P}_t\}$. With outer reference point r , the hypervolume contribution of the cluster is

$$\Delta HV_k = \max(0, HV_r(S) - HV_r(S \setminus S_k)). \quad (3.21)$$

The heuristic scores in this section are the two-dimensional evaluator outputs $s = E(h) = (-HV_{\text{inner}}(h), T(h))$. Therefore, s_1 represents negated inner hypervolume and s_2

represents runtime. Because the first score coordinate is negative inner HV, a simple quality proxy for solution strength is

$$I_k = \max \left(0, -\min_{s \in S_k} s_1 \right), \quad (3.22)$$

with a finite cap in the implementation to avoid an excessively large prior. A runtime proxy is defined as

$$T_k = \frac{1}{1 + \min_{s \in S_k} s_2}. \quad (3.23)$$

The unnormalized prior is

$$\tilde{q}_k = \Delta HV_k + 0.25 I_k + T_k, \quad (3.24)$$

and the normalized prior used by the bandit is

$$q_k = \frac{\tilde{q}_k}{\max_j \tilde{q}_j} \quad (3.25)$$

when the denominator is positive. This design is implemented in `cluster_manager.py`. It is more aligned with the final outer-front objective than a one-dimensional score proxy because it rewards clusters that currently contribute to the heuristic Pareto front while still accounting for inner solution quality and runtime.

3.8 Reward and Credit Assignment

3.8.1 Reward Components

The reward function converts the outcome of one LLM generation call into a scalar signal for the two bandits. For a valid generated heuristic with score s , the reward is

$$R(s) = w_q Q(s) + w_d \max(0, \Delta D) + w_r \text{Rank}(s). \quad (3.26)$$

If the generated heuristic is invalid, cannot be parsed, fails evaluation, or returns a non-finite score, the reward is

$$R(s) = -\lambda_{\text{pen}}. \quad (3.27)$$

The default values in the current implementation are $w_q = 1.0$, $w_d = 0.3$, $w_r = 0.2$, and $\lambda_{\text{pen}} = 1.0$. These settings are exposed through the command-line interface for the weights and fixed in `RewardComputer` for the penalty.

3.8.2 Quality Signal

Let F be the set of valid population scores before the new candidate is inserted. The immediate hypervolume improvement is

$$\text{HVI}(s; F) = \max(0, \text{HV}_r(F \cup \{s\}) - \text{HV}_r(F)). \quad (3.28)$$

The implementation also computes a managed-population HV delta. Let $\text{Manage}_N(\cdot)$ denote the same score-only survivor-selection logic used by the managed population with cap N . Then

$$\Delta \text{HV}_{\text{managed}} = \max(0, \text{HV}_r(\text{Manage}_N(F \cup \{s\})) - \text{HV}_r(\text{Manage}_N(F))). \quad (3.29)$$

Both quantities are normalized by rolling maxima over a recent window. Let $\widehat{\text{HVI}}$ and $\widehat{\Delta \text{HV}}_{\text{managed}}$ denote the normalized values. The quality mode is then

$$Q(s) = \begin{cases} \widehat{\text{HVI}}(s; F), & \text{mode} = \text{dense}, \\ \frac{1}{2}\widehat{\text{HVI}}(s; F) + \frac{1}{2}\widehat{\Delta \text{HV}}_{\text{managed}}(s; F), & \text{mode} = \text{hybrid}, \\ \widehat{\Delta \text{HV}}_{\text{managed}}(s; F), & \text{mode} = \text{final_hv}. \end{cases} \quad (3.30)$$

The default `full` method uses `final_hv`. The ablations `dense_reward` and `hybrid_reward` change only this quality signal while retaining the same BMAB pipeline. This design makes it possible to isolate whether terminal managed-population HV, immediate HVI, or a mixture of the two provides the most useful credit signal.

3.8.3 Diversity and Rank Smoothing

Pure HVI is often sparse: many valid offspring do not immediately expand the outer Pareto front. To provide a denser signal, BMAB-LLM adds rank and diversity terms. The cumulative diversity index used in the reward is the mean pairwise Euclidean distance among valid score vectors:

$$D(F) = \frac{1}{|F|(|F| - 1)} \sum_{i \neq j} \|s_i - s_j\|_2. \quad (3.31)$$

The diversity increment is $\Delta D = D(F \cup \{s\}) - D(F)$. Only positive increments are rewarded. The rank score is

$$\text{Rank}(s) = \frac{1}{|F|} |\{u \in F : \exists j, s_j < u_j\}|, \quad (3.32)$$

In this equation, s and u are heuristic scores of the form $E(h) = (-HV_{\text{inner}}(h), T(h))$. The first coordinate measures negated inner hypervolume, and the second coordinate measures runtime. Since both coordinates are minimized in the outer heuristic population, a smaller coordinate value indicates either higher inner solution quality or lower execution time. The rank term therefore rewards a candidate that improves at least one objective relative to existing population members, even when it does not immediately increase HV.

3.9 Non-Stationarity and Page–Hinkley Reset

The reward distribution in evolutionary heuristic design is non-stationary. A cluster that is useful early may become unproductive after its design pattern has already been exploited. To reduce overconfidence in stale arm statistics, each cluster-level arm is monitored by a Page–Hinkley detector [13].

For a reward sequence (r_t) of one arm, the implementation maintains the running mean $\bar{r}_t$ and cumulative statistic

$$\bar{r}_t = \bar{r}_{t-1} + \frac{r_t - \bar{r}_{t-1}}{t}, \quad (3.33)$$

$$m_t = m_{t-1} + r_t - \bar{r}_t + \delta, \quad (3.34)$$

$$M_t = \max(M_{t-1}, m_t). \quad (3.35)$$

Downward drift is declared when

$$M_t - m_t > \lambda_{\text{PH}}. \quad (3.36)$$

When drift is detected, the corresponding arm statistics are reset to an optimistic prior for the current generation. This mechanism is implemented in `bandit.py`. It is limited to the cluster-level bandit because cluster rewards are local to the current generation and most sensitive to rapid changes in population state.

3.10 Parent Selection and Variation Workflow

The original MPaGE strategy uses intra-cluster mutation and inter-cluster crossover. BMAB-LLM preserves this logic but changes how parents are sampled. The loop first obtains PFG-selected elites through `Population.selection`. It then expands the clustering pool to the full current managed population to give the cluster bandit a richer set of semantic groups. To prevent the wider pool from diluting the PFG signal, parent sampling gives candidates that also appear in the PFG-selected elite set a larger weight:

$$w(h) = \begin{cases} 3, & h \in E_{\text{PFG}}, \\ 1, & \text{otherwise.} \end{cases} \quad (3.37)$$

For mutation, a single parent is sampled from the selected cluster using these weights. For crossover, the first parent is sampled from the selected cluster and the second parent is sampled from other clusters when possible. This preserves semantic recombination while biasing the process toward strong PFG representatives. In the implementation, this logic is used when constructing the mutation or crossover prompt and when sampling parents from the selected semantic groups.

3.11 Complete Algorithm

Algorithm 3.1 gives the high-level workflow implemented by `BMABLLM.run`. The pseudocode abstracts away exception handling and API-specific details, but preserves the execution order that matters methodologically: initialization, budget-aware clustering, adaptive operator and cluster selection, reward computation, bandit update, drift handling, and final population flushing.

Listing 3.1: High-level BMAB-LLM workflow.

```

Algorithm BMAB-LLM
Input: evaluator E, generator L, clusterer C,
      budget B, population cap N, reference point r
Output: managed heuristic population P

initialize BudgetTracker(B), Population P
initialize OperatorBandit, ClusterBandit, RewardComputer

while initialization target is not reached
  and one generation call is affordable do
    charge one budget unit
    h ← L(I1 initialization prompt)
    score ← E(h)
    if score is valid then
      register h in P
    end if
  end while

record the initial budget-HV point

```

```

while budget remains do
  E_pfg <- PFG-selected elites from P
  H_full <- current managed population

  if a clustering call and a generation call are affordable then
    charge one budget unit
    clusters <- C(H_full)
    if clusters are invalid then
      clusters <- singleton fallback clusters
    end if
  else
    clusters <- singleton fallback clusters
  end if

  q <- front-aware cluster priors
  p <- operator priors from OperatorBandit
  reset ClusterBandit using clusters, q, and p

  produced <- 0
  while produced < N
    and one generation call is affordable do
      op <- select mutation or crossover
      arm <- select a cluster arm for op
      parents <- weighted parent sampling from clusters
      prompt <- MPaGE mutation or crossover prompt

      F_before <- scores in P and pending valid offspring
      D_before <- diversity(F_before)

      charge one budget unit
      h_child <- L(prompt)
      score <- E(h_child)
      if score is valid then
        register h_child as pending
      end if

      F_after <- scores in P and pending valid offspring
      D_after <- diversity(F_after)
      reward <- RewardComputer(score, F_before,
                                D_before, D_after)
    end do
    produced <- produced + 1
  end while
end while

```

```

        update OperatorBandit(op, reward)
        update ClusterBandit(arm, reward)
        apply Page-Hinkley reset if drift is detected
        produced <- produced + 1
    end while

    merge pending valid offspring into P when needed
    record bandit state and budget-HV point
end while

flush all pending valid offspring into P
write budget history, budget curve, bandit log, and AUBC

```

3.12 Baseline and Ablation Design

The baseline `mpage_orig` is not a new algorithm. It is a wrapper around the original MPaGE driver that records budget consumption and budget-HV curves using the same profiler infrastructure as BMAB-LLM. This is implemented in `mpage_orig.py`. The purpose is to compare allocation policies while keeping the task evaluators and generated artifacts as consistent as possible.

The current CLI defines the following method variants in `main.py`:

Bảng 3.3: Main method variants relevant to the proposed method.

Variant	Interpretation
<code>full</code>	Proposed BMAB method with the <code>final_hv</code> reward mode, operator bandit, cluster bandit, Page–Hinkley, budget annealing, and PFG-weighted parent sampling.
<code>dense_reward</code>	BMAB pipeline with the immediate-HVI quality signal used for reward ablation.
<code>hybrid_reward</code>	BMAB pipeline with a half immediate-HVI and half managed-final-HV quality signal.
<code>no_budget_anneal</code>	Same as <code>full</code> , but without remaining-budget scaling in the cluster UCB exploration term.
<code>no_ph</code>	Disables practical Page–Hinkley resets by using an extremely large threshold.
<code>no_diversity</code>	Removes the CDI diversity term by setting its weight to zero.
<code>op_only</code>	Disables the cluster bandit and samples clusters uniformly.
<code>cluster_only</code>	Disables the operator bandit and uses a round-robin operator sequence.
<code>mpage_orig</code>	Budget-instrumented wrapper around the original MPaGE implementation.

These variants are important because they separate the contribution of each methodological component. In particular, `dense_reward` and `hybrid_reward` vary only the reward quality mode, while `no_budget_anneal`, `no_ph`, `no_diversity`, `op_only`, and `cluster_only` isolate specific control mechanisms.

Chương 4

Triển khai hệ thống

4.1 Cấu trúc mã nguồn

Kho dự án gồm hai phần chính. Phần thứ nhất là mã nguồn MPaGE gốc ở thư mục `llm4ad/`, kèm entrypoint `main.py` và tài liệu `README.md`. Phần thứ hai là dự án mở rộng `mpage_bmab/`, trong đó có mã nguồn BMAB, tài liệu thiết kế và pipeline thực nghiệm.

Trong `mpage_bmab/`, các tệp quan trọng gồm:

- `bmab_llm.py`: vòng lặp chính của BMAB-LLM.
- `bandit.py`: UCB chọn toán tử, Budgeted UCB chọn cụm và Page–Hinkley.
- `budget.py`: theo dõi ngân sách lời gọi LLM.
- `cluster_manager.py`: gọi LLM để phân cụm ngữ nghĩa và xử lý fallback.
- `reward.py`: tính hypervolume, diversity và reward.
- `profiler.py`: ghi budget curve, AUBC, bandit log và budget history.
- `mpage_orig.py`: wrapper baseline MPaGE trong cùng giao thức ngân sách.
- `experiments/`: cấu hình, chạy, tổng hợp, so sánh và sinh hình cho thực nghiệm.

4.2 Môi trường phần mềm

`requirements.txt` ở thư mục gốc và `mpage_bmab/requirements.txt` liệt kê các phụ thuộc Python. Những phụ thuộc quan trọng gồm `numpy`, `scipy`, `pymoo`, `matplotlib`, `pandas`, `tqdm` và thư viện OpenAI-compatible client. `mpage_bmab/README.md` cho thấy CLI mặc định dùng mô hình `gpt-4o-mini`, còn khóa API được đọc từ các tệp

cấu hình cục bộ trong `mpage_bmab/main.py`. Nội dung các tệp khóa không được sử dụng trong luận văn.

Thông tin phần cứng, GPU, hệ điều hành cụ thể của máy chạy thí nghiệm và chi phí tiền tệ trên mỗi lời gọi không xuất hiện trong kết quả hiện tại. Không tìm thấy thông tin trong mã nguồn hoặc tài liệu của dự án.

4.3 Chuẩn bị bài toán đánh giá

Bốn bài toán trong thực nghiệm hiện tại đều nằm dưới `llm4ad/task/optimization/`. Mỗi bài toán định nghĩa một evaluator và template hàm `select_neighbor`.

Bi-TSP dùng bốn instance, kích thước bài toán 20, timeout 60 giây, archive ban đầu 100 tour ngẫu nhiên và 2000 vòng lặp. Các thông tin này nằm trong `llm4ad/task/optimization/bi_tsp/evaluation.py`. Tri-TSP dùng 20 instance, kích thước 20, timeout 90 giây, archive 100 tour và 20000 vòng lặp, theo `llm4ad/task/optimization/tri_tsp/evaluation.py`. Bi-CVRP dùng tám instance, kích thước 100, capacity 50, timeout 90 giây và 6000 vòng lặp, theo `llm4ad/task/optimization/bi_cvrp/evaluation.py`. Bi-KP dùng tám instance, kích thước 200, capacity 25, timeout 90 giây và 8000 vòng lặp, theo `llm4ad/task/optimization/bi_kp/evaluation.py`.

Các tệp `get_instance.py` trong từng thư mục bài toán đặt seed NumPy bằng 2025, do đó instance benchmark cố định theo mã nguồn hiện tại. Seed 2025–2029 trong thí nghiệm chủ yếu tác động tới vòng lặp sinh/chọn heuristic, không tạo bộ instance mới.

4.4 Pipeline sinh và đánh giá heuristic

Mỗi heuristic được biểu diễn như một đoạn code Python chứa hàm `select_neighbor`. LLM sinh code theo prompt, bộ parser trích xuất hàm, sau đó `SecureEvaluator` trong `llm4ad/base/evaluate.py` chạy evaluator trong tiến trình riêng. Nếu code lỗi, timeout hoặc trả về kết quả không hợp lệ, score có thể là `None`.

Các evaluator trả về score hai chiều ở cấp heuristic. Với Tri-TSP, dù bài toán bên trong có ba mục tiêu, score outer-loop vẫn là hai chiều ($-HV, runtime$); điều này được thể hiện trong `llm4ad/task/optimization/tri_tsp/evaluation.py` và được phản ánh trong `mpage_bmab/main.py`, nơi `ref_point` cho Tri-TSP là `(20.0, 60.0)`.

4.5 Cấu hình thực nghiệm

`mpage_bmab/experiments/configs.py` định nghĩa bốn bài toán, bốn ngân sách và

năm seed. Hàm `pop_size_for` đặt kích thước quần thể theo ngân sách: với các bài toán hai mục tiêu thông thường là 4, 6, 8, 10 cho ngân sách 25, 50, 100, 200; với Tri-TSP cộng thêm 2, tức 6, 8, 10, 12. Cấu hình này cũng được ghi trong `mpage_bmab/documents/HYPERPARAMETERS.md`.

Các tham số bandit mặc định gồm $c_{op} = 1.0$, $c_{cluster} = 1.0$, $\gamma_{budget} = 0.5$, prior count 1 và prior reward 0.5. Reward dùng $w_q = 1.0$, $w_d = 0.3$, $w_r = 0.2$, penalty 1.0 và rank window 50. Page–Hinkley dùng $\delta = 0.005$ và threshold 0.5. Các giá trị này lấy từ `mpage_bmab/documents/HYPERPARAMETERS.md`.

4.6 Script chạy, tổng hợp và so sánh

Pipeline thực nghiệm nằm trong `mpage_bmab/experiments/.run.py` nhận cấu hình, gọi endpoint phù hợp và lưu kết quả theo cấu trúc thư mục. `aggregate.py` duyệt các thư mục kết quả, đọc `budget_curve.json` và `aubc.json`, sau đó tạo `summary.csv`. `compare.py` thực hiện so sánh theo cặp giữa baseline và phương pháp đầy đủ cho từng bài toán, ngân sách và metric.

Các hình trong `mpage_bmab/experiments/results/images/` được sinh bởi `mpage_bmab/experiments/results/image_generation_code/generate_result_images.py`. Script này tạo các biểu đồ AUBC, HV cuối cùng, budget curves, invalid/null-score rate và valid heuristic yield. Luận văn sao chép các hình này vào `thesis/figures/`.

4.7 Artifact đầu ra

Mỗi lượt chạy có thể tạo các tệp sau:

- `samples/`: các heuristic được lưu, kèm code và score nếu có.
- `population/`: quần thể heuristic.
- `budget_curve.json`: chuỗi điểm ($budget_consumed, hv, pareto_size$).
- `budget_history.json`: lịch sử tiêu thụ ngân sách.
- `bandit_log.json`: quyết định bandit và reward.
- `aubc.json`: AUBC cuối cùng.
- `run_log.txt`: log quá trình chạy.

Cấu trúc artifact được mô tả trong `mpage_bmab/README.md`, `mpage_bmab/experiments/README.md` và `mpage_bmab/profiler.py`.

4.8 Tái lập kết quả

Theo `mpage_bmab/experiments/README.md`, có thể chạy các suite như `smoke`, `headline`, `budget50`, `full`, `hvf`, `fix_smoke`, `hv_final_priority` và `hv_final_full`. Bộ kết quả đang phân tích trong Chương 5 tương ứng đầy đủ với hai phương pháp, bốn bài toán, bốn ngân sách và năm seed. Các lệnh cụ thể có thể thay đổi tùy môi trường API, nhưng entrypoint chính là `mpage_bmab/experiments/run.py`.

Vì hệ thống phụ thuộc LLM qua API, tái lập bit-by-bit là khó nếu dịch vụ LLM thay đổi hoặc sampling không hoàn toàn quyết định. Kho mã nguồn giảm rủi ro bằng cách lưu toàn bộ samples, curves và logs. Tuy nhiên, không tìm thấy thông tin về snapshot chính xác của model API, cấu hình temperature đầy đủ ở mọi nơi, hoặc chi phí token thực tế trong kết quả hiện tại.

4.9 Các điểm cần thận trọng trong triển khai

Có ba điểm cần đọc cẩn thận khi diễn giải kết quả. Thứ nhất, `budget_history` của `mpage_orig` được ghi ở dạng tổng hợp trong wrapper, còn BMAB ghi chi tiết hơn; vì vậy số dòng lịch sử không nên được so sánh như số event tương đương. Thứ hai, BMAB chỉ lưu samples hợp lệ, còn MPaGE-orig có thể lưu score `None`; do đó `invalid/null-score rate` là một chẩn đoán chi phí, không phải metric lưu trữ giống hệt. Thứ ba, các tệp `paras.yaml` của benchmark có giá trị `timeout` khác với evaluator Python trong một số trường hợp; luận văn ưu tiên mã evaluator vì đó là đường thực thi thực tế.

Chương 5

Thực nghiệm và đánh giá

5.1 Câu hỏi thực nghiệm

Thực nghiệm hiện tại nhằm trả lời ba câu hỏi trên bộ kết quả hiện có. Thứ nhất, `mpage_bmab` có dùng ngân sách lời gọi LLM hiệu quả hơn MPaGE-orig hay không, xét theo AUBC? Thứ hai, hiệu quả theo ngân sách có đi kèm cải thiện HV cuối cùng hay không? Thứ ba, tỷ lệ heuristic hợp lệ và invalid/null-score có giúp giải thích khác biệt giữa hai phương pháp hay không?

Các câu hỏi này được đặt dựa trên mục tiêu trong `mpage_bmab/documents/IDEA.md`, metric trong `mpage_bmab/documents/METRICS_AUBC_HV.md` và kết quả trong `mpage_bmab/experiments/results/`.

5.2 Thiết lập thực nghiệm

Bộ kết quả hiện tại gồm 160 lượt chạy: hai phương pháp, bốn bài toán, bốn mức ngân sách và năm seed. Hai phương pháp là `full` và `mpage_orig`. Trong phần thảo luận, `full` được gọi là `mpage_bmab` hoặc BMAB; `mpage_orig` là MPaGE gốc chạy qua wrapper ngân sách.

Các bài toán gồm Bi-TSP, Tri-TSP, Bi-CVRP và Bi-KP. Các ngân sách là $B \in \{25, 50, 100, 200\}$. Các seed là 2025, 2026, 2027, 2028 và 2029. Mỗi lượt chạy đều tiêu thụ đúng ngân sách được cấu hình, theo `mpage_bmab/experiments/results/summary.csv` và kiểm tra trong `mpage_bmab/experiments/results/analysis_report.md`.

Các số liệu trong chương này tập trung vào hai phương pháp có kết quả đầy đủ trong thư mục thực nghiệm hiện có. Các ablation như `dense_reward`, `hybrid_reward`, `no_budget_anneal`, `no_ph`, `no_diversity`, `op_only` và `cluster_only` cần được chạy bổ sung nếu muốn định lượng riêng đóng góp của từng thành phần.

5.3 Metric đánh giá

Metric chính là AUBC. Metric này đo diện tích trung bình dưới đường cong HV theo ngân sách, nên phản ánh mức độ đạt chất lượng tốt sớm trong quá trình tối ưu. Metric phụ là `hv_final`, được định nghĩa trong `mpage_bmab/experiments/aggregate.py` là giá trị `h` cuối cùng trong `budget_curve.json`. Ngoài ra, luận văn dùng hai chỉ số chẩn đoán: `invalid/null-score rate` và `valid heuristic yield per budget`.

Với budget curve, trục hoành trong các hình được chuẩn hóa bằng `budget_consumed/total_budget`. Trục tung là HV của quần thể heuristic tại thời điểm ngân sách tương ứng, được tính trong `mpage_bmab/profiler.py`. Do đó, về mặt định nghĩa, điểm ở $x = 1$ của một lượt chạy riêng lẻ chính là HV cuối cùng của lượt chạy đó nếu curve có điểm tại ngân sách cuối. Tuy nhiên, trong hình budget curve, script vẽ hình nội suy và trung bình các đường cong theo seed sau khi chuẩn hóa, theo `mpage_bmab/experiments/results/image_generation_code/generate_result_images.py`. Vì vậy giá trị trục quan ở $x = 1$ nên gần với trung bình `hv_final`, nhưng có thể khác nhẹ do nội suy, khử trùng lặp điểm ngân sách và cách render đường trung bình. Nếu khác lớn, cần kiểm tra code aggregation hoặc curve source.

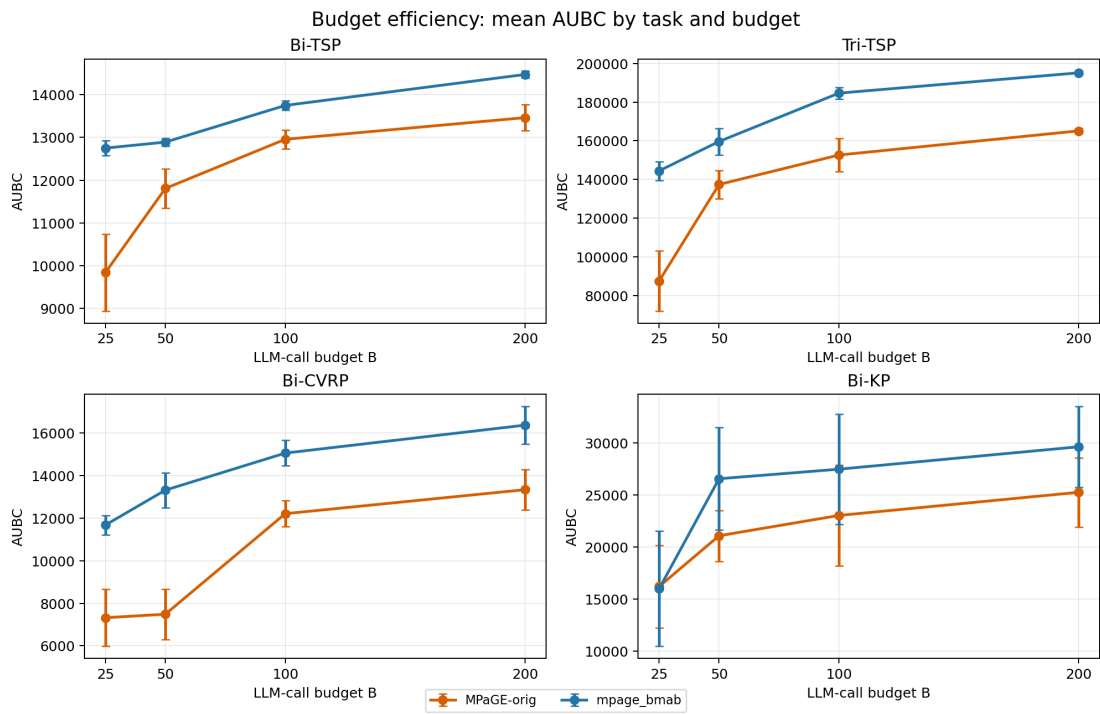
5.4 Kết quả AUBC

Bảng 5.1 và Hình 5.1 cho thấy BMAB đạt AUBC trung bình cao hơn MPaGE-orig trong 15/16 tổ hợp bài toán–ngân sách. Tổng hợp theo cặp seed, BMAB thắng 71/80 cặp. Mức cải thiện trung bình theo cell là khoảng 26.1%, theo `mpage_bmab/experiments/results/analysis_report.md`.

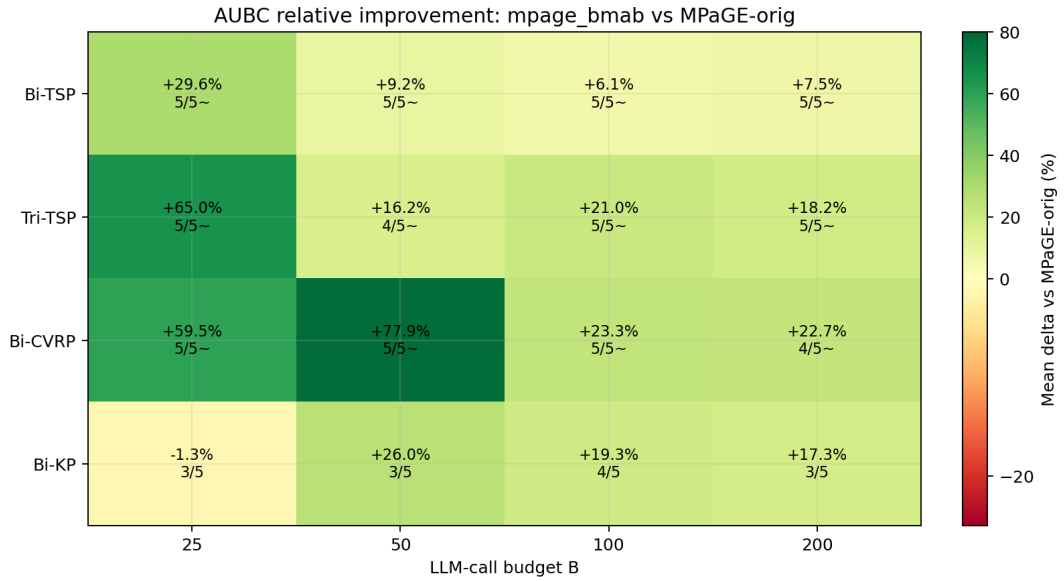
Bảng 5.1: So sánh AUBC trung bình giữa `mpage_bmab full` và `mpage_orig`. Nguồn số liệu: `mpage_bmab/experiments/results/derived_cell_stats.csv` và `mpage_bmab/experiments/results/comparisons_aubc.csv`.

Bài toán	B	BMAB	MPaGE-orig	$\Delta\%$	Thắng / 5
Bi-TSP	25	12750.8	9841.9	29.6	5
Bi-TSP	50	12890.6	11807.6	9.2	5
Bi-TSP	100	13748.1	12956.3	6.1	5
Bi-TSP	200	14471.8	13462.6	7.5	5
Tri-TSP	25	144482.9	87564.7	65.0	5
Tri-TSP	50	159646.0	137434.6	16.2	4
Tri-TSP	100	184626.5	152646.7	21.0	5
Tri-TSP	200	195125.3	165126.2	18.2	5

Bài toán	B	BMAB	MPaGE-orig	$\Delta\%$	Thắng / 5
Bi-CVRP	25	11679.8	7322.7	59.5	5
Bi-CVRP	50	13318.3	7486.2	77.9	5
Bi-CVRP	100	15059.8	12211.7	23.3	5
Bi-CVRP	200	16372.1	13339.4	22.7	4
Bi-KP	25	16007.4	16217.4	-1.3	3
Bi-KP	50	26573.3	21096.1	26.0	3
Bi-KP	100	27489.8	23046.1	19.3	4
Bi-KP	200	29643.5	25270.8	17.3	3



Hình 5.1: AUBC trung bình theo ngân sách. Nguồn: [mpage_bmab/experiments/results/images/aubc_mean_by_budget.png](https://mpage_bmab.github.io/experiments/results/images/aubc_mean_by_budget.png).



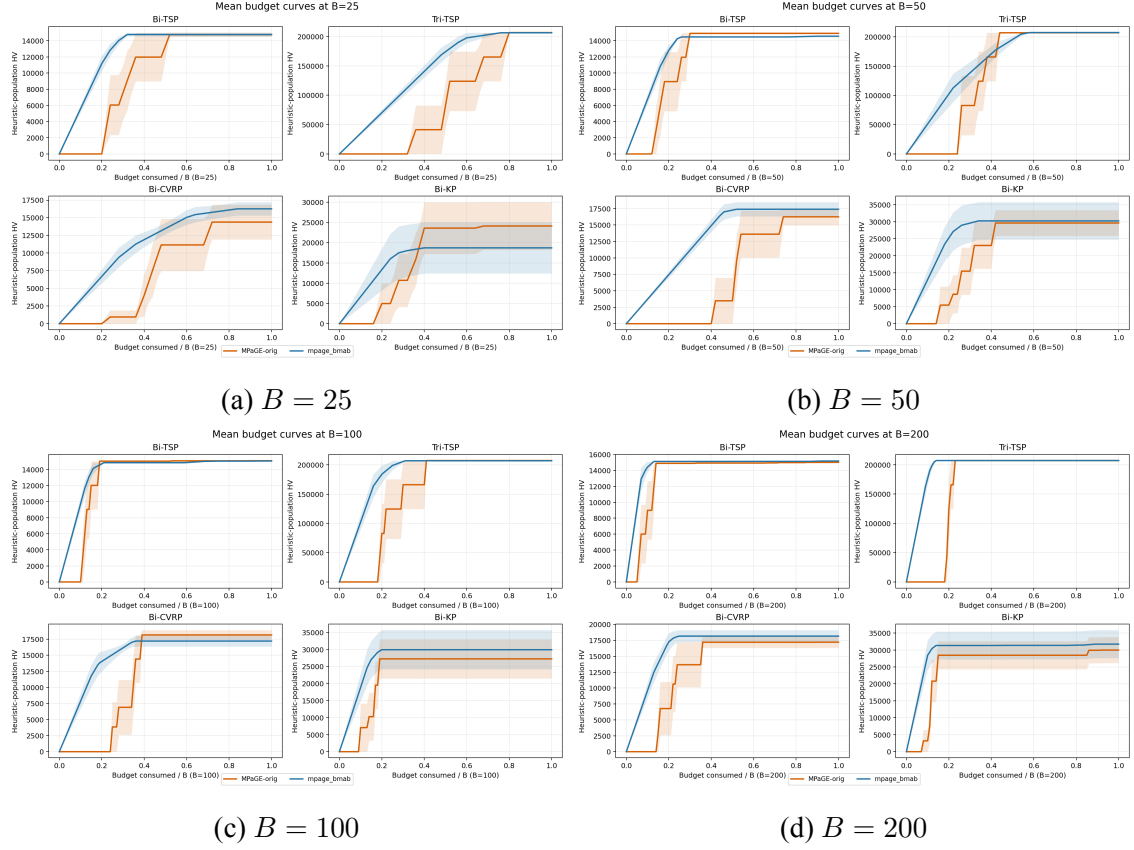
Hình 5.2: Heatmap phần trăm thay đổi AUBC của BMAB so với MPaGE-orig. Nguồn: `mpage_bmab/experiments/results/images/aubc_delta_percent_heatmap.png`.

Kết quả nổi bật nhất nằm ở Bi-CVRP và Tri-TSP. Với Bi-CVRP, BMAB cải thiện AUBC lần lượt 59.5%, 77.9%, 23.3% và 22.7% khi $B = 25, 50, 100, 200$. Với Tri-TSP, cải thiện tương ứng là 65.0%, 16.2%, 21.0% và 18.2%. Hai bài toán này có chi phí đánh giá và không gian heuristic phức tạp hơn Bi-TSP; do đó khả năng điều phối lời gọi LLM sớm có thể tạo lợi thế rõ hơn.

Bi-TSP có mức cải thiện nhỏ hơn nhưng rất ổn định: cả bốn ngân sách đều thắng 5/5 seed. Bi-KP là trường hợp ít ổn định nhất. Tại $B = 25$, BMAB thấp hơn MPaGE-orig 1.3%, còn các ngân sách lớn hơn đều dương. Điều này gợi ý rằng với Bi-KP, giai đoạn đầu có thể nhạy với heuristic khởi tạo hoặc reward chưa đủ phân biệt ở ngân sách rất nhỏ.

5.5 Đường cong ngân sách

Các đường cong ngân sách ở Hình 5.3 giải thích vì sao AUBC tăng mạnh ngay cả khi HV cuối không luôn vượt trội. BMAB thường nâng HV sớm hơn, đặc biệt ở ngân sách nhỏ và trung bình. Khi đường cong tăng nhanh trong nửa đầu ngân sách, diện tích dưới đường cong tăng, kể cả khi điểm cuối gần bằng baseline.



Hình 5.3: Đường cong HV theo ngân sách chuẩn hóa. Nguồn: các tệp `budget_curves_B*.png` trong `mpage_bmab/experiments/results/images/`.

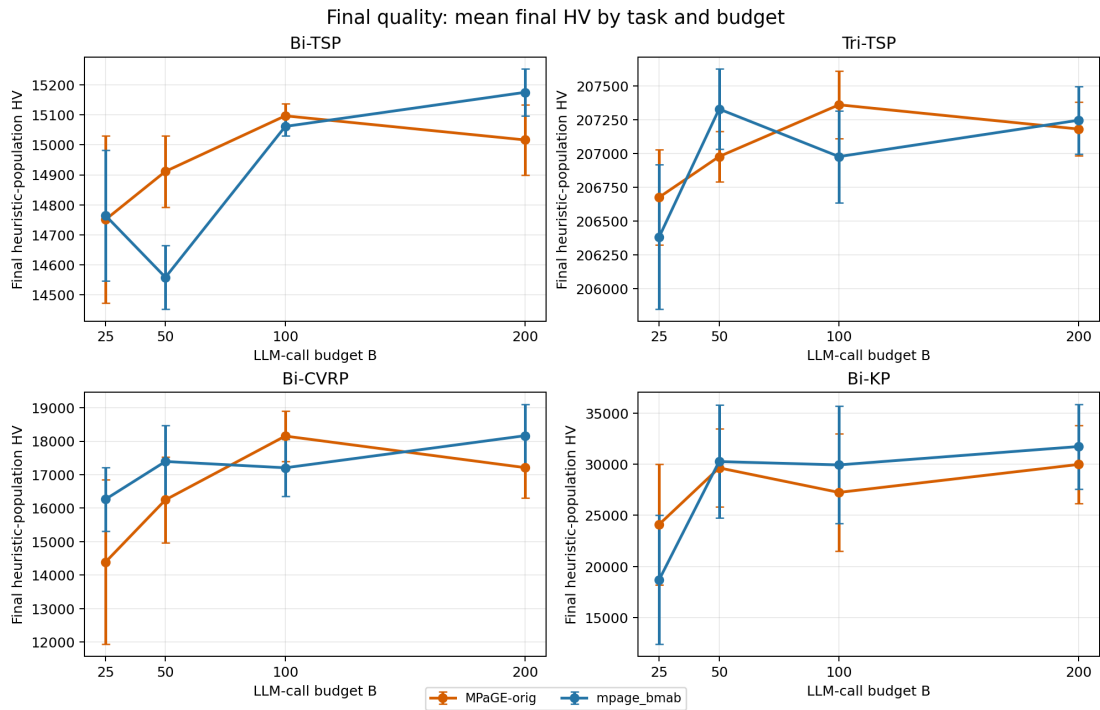
Các đường cong cũng cho thấy hiệu ứng giảm dần. Khi ngân sách tăng lên, cả hai phương pháp đều có thêm cơ hội tìm heuristic tốt; khoảng cách AUBC ở Bi-TSP giảm từ 29.6% tại $B = 25$ xuống 6.1–9.2% tại các ngân sách lớn hơn. Điều này phù hợp với trực giác bandit: lợi thế lớn nhất của điều phối thích nghi xuất hiện khi tài nguyên khan hiếm.

5.6 HV cuối cùng

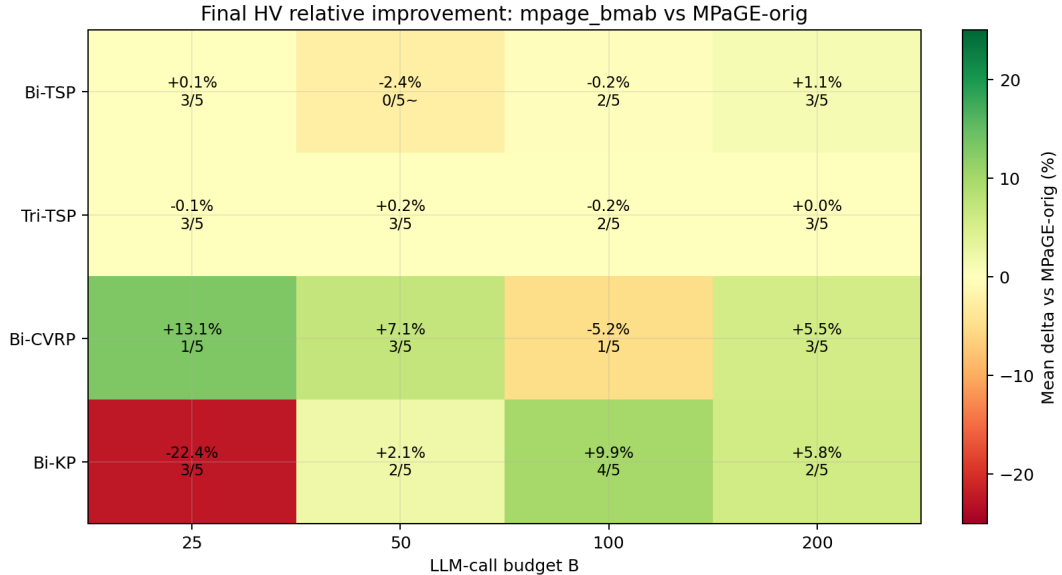
Bảng 5.2 và Hình 5.4 cho thấy kết quả HV cuối cùng ít nhất quán hơn AUBC. BMAB cao hơn trong 10/16 cell và thắng 38/80 cặp seed. Mức cải thiện trung bình theo cell chỉ khoảng 0.9%, theo `mpage_bmab/experiments/results/analysis_report.md`.

Bảng 5.2: So sánh HV cuối cùng trung bình cho cấu hình full. HV ở đây là hypervolume của quần thể heuristic trong quá trình thiết kế, không phải HV nghiệm của từng bài toán con trong bài báo gốc. Nguồn số liệu: mpage_bmab/experiments/results/derived_cell_stats.csv và mpage_bmab/experiments/results/comparisons_hv_final.csv.

Bài toán	B	BMAB	MPaGE-orig	$\Delta\%$	Thắng / 5
Bi-TSP	25	14764.5	14750.8	0.1	3
Bi-TSP	50	14558.5	14911.4	-2.4	0
Bi-TSP	100	15061.3	15096.6	-0.2	2
Bi-TSP	200	15174.9	15016.3	1.1	3
Tri-TSP	25	206382.3	206676.6	-0.1	3
Tri-TSP	50	207330.8	206978.3	0.2	3
Tri-TSP	100	206978.0	207362.1	-0.2	2
Tri-TSP	200	207247.8	207182.3	0.0	3
Bi-CVRP	25	16264.4	14386.8	13.1	1
Bi-CVRP	50	17393.6	16246.9	7.1	3
Bi-CVRP	100	17205.0	18153.3	-5.2	1
Bi-CVRP	200	18164.2	17210.4	5.5	3
Bi-KP	25	18721.8	24123.1	-22.4	3
Bi-KP	50	30255.6	29638.7	2.1	2
Bi-KP	100	29927.3	27239.9	9.9	4
Bi-KP	200	31724.1	29974.8	5.8	2



Hình 5.4: HV cuối cùng trung bình theo ngân sách. Nguồn: `mpage_bmab/experiments/results/images/hv_final_mean_by_budget.png`.



Hình 5.5: Heatmap phần trăm thay đổi HV cuối cùng của BMAB so với MPaGE-orig. Nguồn: `mpage_bmab/experiments/results/images/hv_final_delta_percent_heatmap.png`.

Điểm quan trọng là HV cuối cùng không phủ nhận kết quả AUBC. Nó cho thấy đóng góp chính của BMAB là đạt chất lượng tốt hơn sớm hơn, chứ không nhất thiết tạo ra Pareto

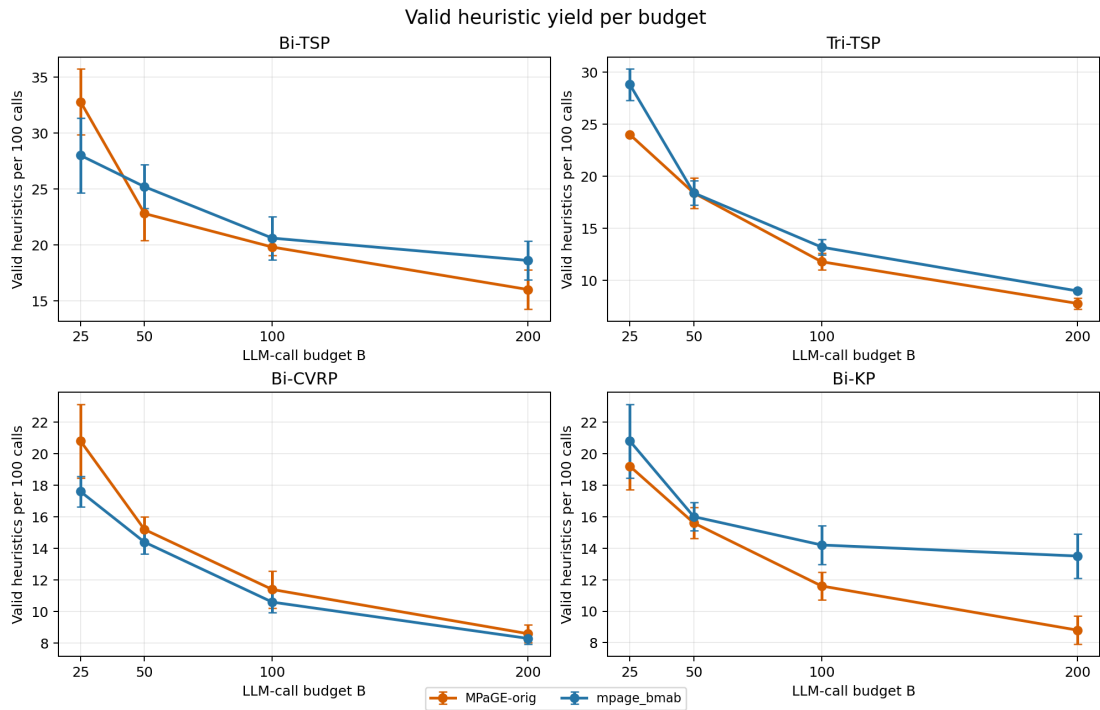
front cuối cùng tốt hơn ở mọi bài toán. Trong bối cảnh lời gọi LLM có chi phí, đây vẫn là một lợi ích thực tế: nếu người dùng dừng sớm hoặc ngân sách nhỏ, AUBC cao có giá trị hơn.

5.7 Tỷ lệ invalid/null-score và valid heuristic yield

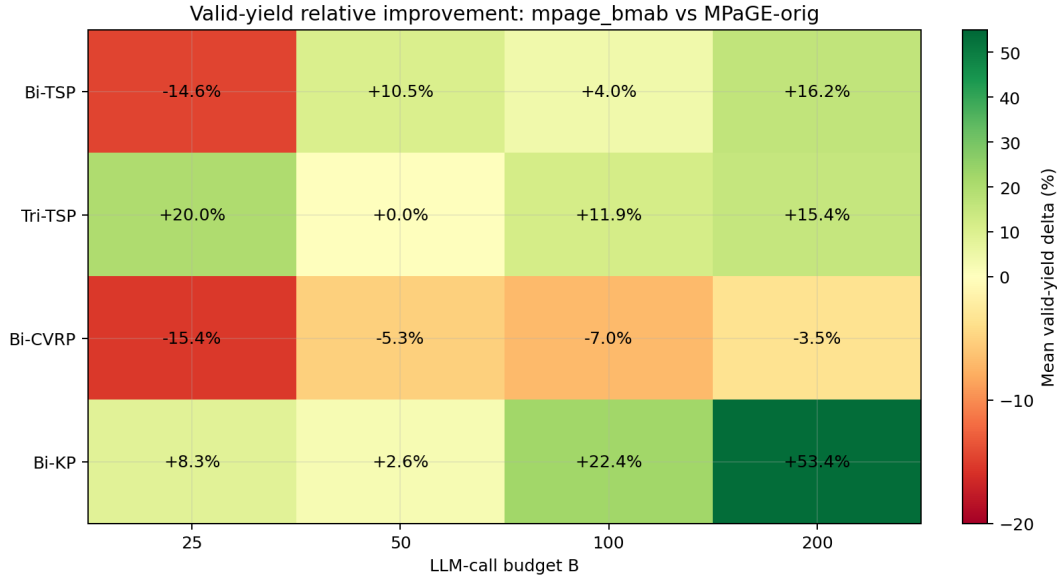
Các chỉ số validity giúp trả lời liệu cải thiện AUBC có đơn thuần đến từ việc sinh được nhiều heuristic hợp lệ hơn hay không. Theo `mpage_bmab/experiments/results/analysis_report.md`, MPaGE-orig lưu 1625 samples, trong đó 986 valid và 639 có score null, tương ứng null-score rate ghi nhận là 39.3%. BMAB lưu 1091 scored samples và không có score null trong samples, nhưng do BMAB chỉ lưu heuristic hợp lệ, invalid attempts phải suy ra từ `budget_history`. Tổng thể, valid yield của BMAB là 14.5 heuristic hợp lệ trên 100 lời gọi, so với 13.1 của MPaGE-orig.

Bảng 5.3: Số heuristic hợp lệ trên 100 lời gọi ngân sách trong bộ kết quả hiện có. Nguồn số liệu: `mpage_bmab/experiments/results/validity_cell_stats.csv` và báo cáo phân tích hiện có trong `mpage_bmab/experiments/results/analysis_report.md`.

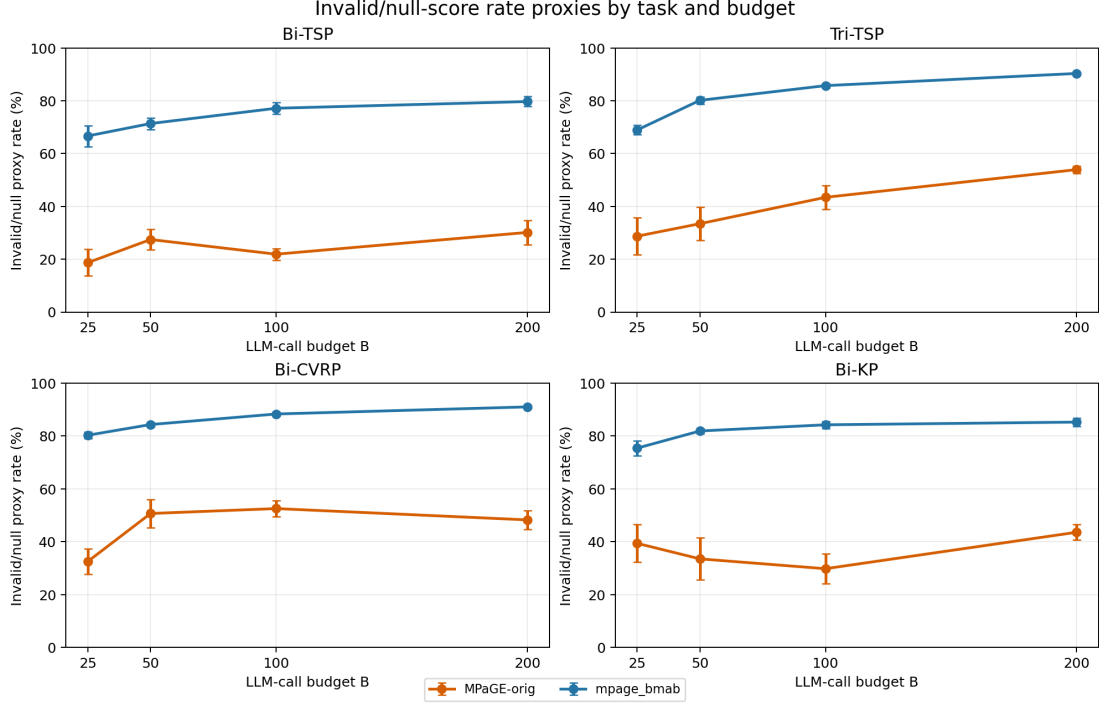
Bài toán	B	BMAB	MPaGE-orig	$\Delta\%$
Bi-TSP	25	28.0	32.8	-14.6
Bi-TSP	50	25.2	22.8	10.5
Bi-TSP	100	20.6	19.8	4.0
Bi-TSP	200	18.6	16.0	16.2
Tri-TSP	25	28.8	24.0	20.0
Tri-TSP	50	18.4	18.4	0.0
Tri-TSP	100	13.2	11.8	11.9
Tri-TSP	200	9.0	7.8	15.4
Bi-CVRP	25	17.6	20.8	-15.4
Bi-CVRP	50	14.4	15.2	-5.3
Bi-CVRP	100	10.6	11.4	-7.0
Bi-CVRP	200	8.3	8.6	-3.5
Bi-KP	25	20.8	19.2	8.3
Bi-KP	50	16.0	15.6	2.6
Bi-KP	100	14.2	11.6	22.4
Bi-KP	200	13.5	8.8	53.4



Hình 5.6: Valid heuristic yield theo ngân sách. Nguồn: `mpage_bmab/experiments/results/images/valid_yield_by_budget.png`.



Hình 5.7: Heatmap phần trăm thay đổi valid heuristic yield. Nguồn: `mpage_bmab/experiments/results/images/valid_yield_delta_percent_heatmap.png`.



Hình 5.8: Tỷ lệ invalid/null-score theo ngân sách. Nguồn: `mpage_bmab/experiments/results/images/invalid_null_rate_by_budget.png`.

Bảng validity cho thấy BMAB không luôn sinh nhiều heuristic hợp lệ hơn. Ở Bi-CVRP, BMAB có valid yield thấp hơn MPaGE-orig ở cả bốn ngân sách, nhưng AUBC lại cao hơn rất nhiều. Đây là bằng chứng quan trọng: lợi thế AUBC của BMAB không thể giải thích chỉ bằng số heuristic hợp lệ. Có nhiều khả năng cơ chế bandit giúp phân bổ lời gọi vào những vùng tạo cải thiện HV lớn hơn, dù tổng số heuristic hợp lệ không cao hơn.

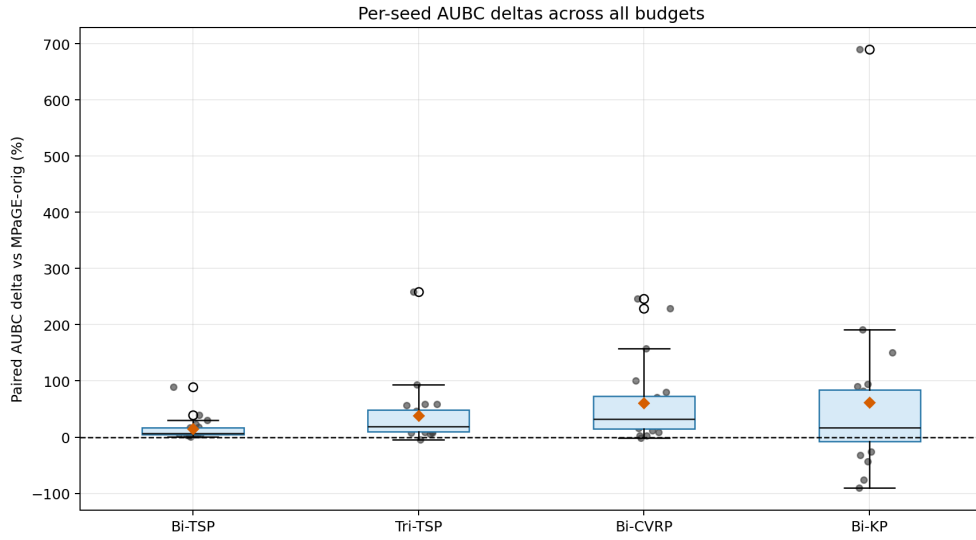
Ngược lại, ở Bi-KP ngân sách lớn, valid yield của BMAB tăng rõ: tại $B = 200$, BMAB đạt 13.5 heuristic hợp lệ trên 100 lời gọi, cao hơn 53.4% so với MPaGE-orig. Điều này có thể góp phần vào cải thiện AUBC 17.3% và HV cuối 5.8% tại cùng ngân sách. Tuy nhiên, đây là diễn giải từ kết quả hiện có, không phải quan hệ nhân quả đã được kiểm định bằng ablation.

5.8 Phân tích thống kê

`mpage_bmab/experiments/compare.py` dùng so sánh theo cặp và kiểm định Wilcoxon signed-rank. Với năm seed, kiểm định hai phía có độ phân giải thô; ngay cả khi một phương pháp thắng cả 5/5 cặp, p nhỏ nhất thường là 0.0625. Vì vậy không có cell nào đạt $p < 0.05$ trong kết quả hiện tại. Luận văn không diễn giải các kết quả này là có ý nghĩa thống kê mạnh.

Tuy nhiên, nhất quán thực nghiệm vẫn đáng chú ý. AUBC thắng 15/16 cell và 71/80 cặp seed là một xu hướng rộng trên nhiều bài toán/ngân sách. Hình 5.9 cho thấy phân phối

chênh lệch AUBC theo cặp seed chủ yếu nằm phía dương, dù có outlier và một số trường hợp âm ở Bi-KP.



Hình 5.9: Phân phối chênh lệch AUBC theo cặp seed. Nguồn: `mpage_bmab/experiments/results/images/paired_aucb_delta_boxplot.png`.

5.9 Diễn giải theo từng nhóm bài toán

Bi-TSP là bài toán có cải thiện AUBC ổn định nhưng biên độ vừa phải. Điều này có thể do không gian heuristic hai mục tiêu đơn giản hơn và MPaGE-orig đã đủ mạnh khi ngân sách lớn. BMAB vẫn có lợi thế ở ngân sách nhỏ vì chọn lời gọi có định hướng hơn.

Tri-TSP có cải thiện AUBC lớn, đặc biệt tại $B = 25$. Sau khi người dùng chạy lại thí nghiệm `mpage_bmab/experiments/results/full/tri_tsp/B25/seed2027`, kết quả hiện tại cho cell này là AUBC BMAB cao hơn 65.0% và thắng 5/5 seed. HV cuối của Tri-TSP lại gần như ngang nhau. Điều này củng cố kết luận rằng BMAB có lợi thế tốc độ đạt chất lượng, không nhất thiết hơn ở điểm kết thúc.

Bi-CVRP là trường hợp thú vị nhất. BMAB có AUBC cao hơn mạnh dù valid yield thấp hơn. Bài toán CVRP có ràng buộc tải trọng và cấu trúc route phức tạp, nên không phải mọi heuristic hợp lệ đều có ích. Bandit có thể ưu tiên các cụm/toán tử tạo heuristic có giá trị biên cao hơn, thay vì chỉ tăng số lượng heuristic hợp lệ.

Bi-KP là trường hợp không ổn định. Ở ngân sách 25, AUBC giảm nhẹ và HV cuối giảm mạnh 22.4%. Điều này có thể xuất phát từ tính nhạy của reward ở giai đoạn đầu hoặc từ việc heuristic tốt cho KP cần một dạng cấu trúc khác so với prompt hiện tại. Không tìm thấy ablation trong kết quả hiện tại để xác nhận nguyên nhân.

5.10 Hạn chế của thực nghiệm

Bộ kết quả hiện tại có bốn hạn chế chính. Thứ nhất, mỗi cell chỉ có năm seed, chưa đủ mạnh cho kết luận thống kê ở mức 0.05. Thứ hai, kết quả chỉ so sánh `full` với `mpage_orig`; các ablation `no_ph`, `no_diversity`, `op_only`, `cluster_only`, `dense_reward`, `hybrid_reward`, `no_budget_anneal` và `mpage_budget` có trong mã nhưng không có đầy đủ trong thư mục kết quả hiện tại. Thứ ba, không có số liệu token, latency API, chi phí tiền tệ, GPU hoặc bộ nhớ. Không tìm thấy thông tin trong mã nguồn hoặc tài liệu của dự án. Thứ tư, metric hiện tại là heuristic-population HV/AUBC, không trực tiếp so sánh với bảng nghiệm chuẩn hóa trong bài báo MPaGE.

5.11 Kết luận thực nghiệm

Bằng chứng thực nghiệm ủng hộ kết luận rằng `mpage_bmab` cải thiện hiệu quả sử dụng ngân sách lời gọi LLM. Kết luận mạnh nhất là về AUBC: BMAB thường đạt HV cao sớm hơn trong quá trình chạy. Kết luận yếu hơn là về HV cuối cùng: kết quả mixed và phụ thuộc bài toán. Validity analysis cho thấy số heuristic hợp lệ là một yếu tố liên quan nhưng không đủ để giải thích toàn bộ cải thiện, đặc biệt ở Bi-CVRP. Các ablation reward và điều phối cần được chạy thêm để định lượng đóng góp riêng của từng cơ chế.

Chương 6

Kết luận và hướng phát triển

6.1 Tổng kết

Luận văn đã phân tích dự án `mpage_bmab`, một phần mở rộng của MPaGE cho bài toán thiết kế heuristic bằng mô hình ngôn ngữ lớn trong tối ưu tổ hợp đa mục tiêu. Dự án giữ lại các thành phần cốt lõi của MPaGE như SEMO, Pareto Front Grid, phân cụm ngữ nghĩa, prompt sinh heuristic và evaluator benchmark, đồng thời thêm một lớp điều phối ngân sách dựa trên bandit.

Từ mã nguồn và tài liệu, phương pháp đề xuất có thể được tóm tắt như sau. Hệ thống xem mỗi lời gọi LLM là một tài nguyên có chi phí, dùng `BudgetTracker` để đảm bảo ngân sách cứng, dùng `OperatorBandit` để chọn giữa đột biến và lai ghép, dùng `ClusterBandit` để chọn cụm ngữ nghĩa trong từng thế hệ, dùng Page–Hinkley để reset thống kê khi phần thưởng trôi, và dùng `RewardComputer` để biến kết quả đánh giá heuristic thành tín hiệu học. Phương pháp chính dùng reward hướng HV cuối cùng, hỗ trợ ablation `dense_reward/hybrid_reward`, annealing thăm dò theo ngân sách và xả quần thể chờ trước khi ghi kết quả cuối.

6.2 Đóng góp chính

Đóng góp đầu tiên là mô hình hóa rõ bài toán thiết kế heuristic dưới ràng buộc ngân sách lời gọi LLM. Thay vì chỉ quan tâm chất lượng cuối cùng, dự án đưa đường cong chất lượng theo ngân sách vào quá trình đánh giá thông qua AUBC.

Đóng góp thứ hai là thiết kế cơ chế bandit hai tầng. Tầng toán tử học khuynh hướng dài hạn giữa mutation và crossover; tầng cụm học lựa chọn ngắn hạn dựa trên chất lượng cụm, prior toán tử và ngân sách còn lại. Thiết kế này phù hợp với bản chất không ổn định của vòng lặp sinh heuristic bằng LLM.

Đóng góp thứ ba là xây dựng pipeline thực nghiệm có thể tổng hợp, so sánh và trực

quan hóa kết quả. Các artifact như `budget_curve.json`, `aubc.json`, `bandit_log.json`, `validity_stats.csv` và các hình trong `mpage_bmab/experiments/results/images/` cho phép phân tích không chỉ điểm cuối mà cả quá trình tiêu thụ ngân sách.

Đóng góp thứ tư là thiết kế các cơ chế hướng HV cuối cùng: reward dùng score thực của heuristic vừa sinh, pending offspring được xả vào quần thể cuối, lời gọi cuối không bị tiêu cho phân cụm đơn thuần, prior cụm xét đóng góp HV/front và chọn cha mẹ có trọng số theo PFG.

6.3 Phát hiện chính

Kết quả hiện tại cho thấy `mpage_bmab` cải thiện AUBC trong 15/16 tổ hợp bài toán–ngân sách và thắng 71/80 cặp chạy theo seed so với `mpage_orig`. Mức cải thiện nổi bật ở Bi-CVRP và Tri-TSP, đặc biệt trong ngân sách nhỏ hoặc trung bình. Điều này phù hợp với giả thuyết rằng điều phối lời gọi LLM thích nghi có lợi khi tài nguyên khan hiếm.

Tuy nhiên, HV cuối cùng trong bộ kết quả hiện có không cải thiện nhất quán. BMAB chỉ cao hơn trong 10/16 cell và thắng 38/80 cặp seed. Vì vậy, kết luận đúng nhất cho bộ kết quả này là BMAB cải thiện hiệu quả theo ngân sách, không phải luôn tạo ra quần thể heuristic cuối cùng tốt hơn.

Phân tích invalid/null-score và valid heuristic yield cho thấy việc sinh heuristic hợp lệ là quan trọng nhưng không đủ để giải thích mọi kết quả. Ở Bi-CVRP, BMAB có valid yield thấp hơn nhưng AUBC cao hơn đáng kể, cho thấy chất lượng phân bổ lời gọi và giá trị biên của heuristic mới có thể quan trọng hơn số lượng heuristic hợp lệ.

6.4 Hạn chế

Luận văn chỉ dựa trên kết quả hiện có trong `mpage_bmab/experiments/results/`. Mỗi cell có năm seed, do đó kiểm định Wilcoxon hai phía không đạt mức $p < 0.05$. Các kết luận thống kê cần được xem là bằng chứng ban đầu có tính nhất quán thực nghiệm, không phải chứng minh cuối cùng.

Bộ kết quả hiện tại không chứa ablation đầy đủ cho từng thành phần như Page–Hinkley, diversity reward, operator-only, cluster-only, reward mode hoặc budget annealing. Vì vậy, chưa thể định lượng riêng đóng góp của từng mô-đun. Không tìm thấy thông tin trong mã nguồn hoặc tài liệu của dự án về GPU, bộ nhớ, chi phí token, chi phí tiền tệ hoặc latency API.

Ngoài ra, kết quả trong luận văn dùng HV/AUBC ở cấp quần thể heuristic. Chúng không tương đương trực tiếp với các bảng benchmark chuẩn hóa trong bài báo MPaGE gốc. Mọi so sánh trong luận văn vì vậy được giới hạn trong cùng pipeline `mpage_bmab`.

6.5 Hướng phát triển

Hướng phát triển quan trọng nhất là chạy thêm các thí nghiệm ablation cho HV cuối cùng. Trong nhóm này, `full` là phương pháp chính với reward `final_hv`, `dense_reward` thay reward về HVI tức thời, `hybrid_reward` trộn hai tín hiệu, và `no_budget_anneal` kiểm tra vai trò của annealing thăm dò theo ngân sách.

Hướng tiếp theo là chạy đầy đủ ablation truyền thống. Các cấu hình `no_ph`, `no_diversity`, `op_only`, `cluster_only` và `mpage_budget` đã xuất hiện trong `mpage_bmab/main.py`, nhưng chưa có kết quả đầy đủ trong thư mục hiện tại. Ablation sẽ giúp trả lời thành phần nào đóng góp nhiều nhất cho AUBC.

Hướng thứ hai là mở rộng số seed và mô hình LLM. Với nhiều seed hơn, kiểm định thống kê sẽ có lực mạnh hơn. Với nhiều model hơn, có thể kiểm tra liệu bandit có còn hiệu quả khi chất lượng sinh mã, độ ổn định và chi phí lời gọi thay đổi.

Hướng thứ ba là đưa chi phí token và latency vào ngân sách. Hiện tại mỗi lời gọi được tính chi phí bằng nhau, nhưng trong thực tế prompt dài, completion dài hoặc model khác nhau có chi phí khác nhau. Một biến thể budgeted MAB theo chi phí token có thể phản ánh thực tế triển khai tốt hơn.

Hướng thứ tư là phân tích sâu hơn lỗi sinh mã. Nếu lưu đầy đủ `invalid attempts` của BMAB cùng loại lỗi, có thể tách lỗi cú pháp, lỗi runtime, timeout và heuristic hợp lệ nhưng kém. Điều này giúp thiết kế prompt hoặc cơ chế sửa lỗi tốt hơn.

Hướng cuối cùng là kết hợp thêm cơ chế tìm kiếm cấp cao như Monte Carlo Tree Search hoặc lập kế hoạch prompt, nhưng hướng này cần được kiểm chứng cẩn thận để tránh làm tăng chi phí LLM vượt quá lợi ích AUBC.

6.6 Kết luận chung

`mpage_bmab` là một mở rộng hợp lý và có bằng chứng thực nghiệm của MPaGE trong bối cảnh ngân sách LLM hữu hạn. Điểm mạnh của dự án nằm ở việc biến ngân sách thành một phần của thuật toán thay vì chỉ là giới hạn cứng. Kết quả hiện có cho thấy cách tiếp cận bandit hai tầng có thể cải thiện đáng kể chất lượng tích lũy theo ngân sách, đặc biệt trên các bài toán phức tạp. Dù còn thiếu ablation và số seed lớn hơn, dự án đã tạo ra một nền tảng rõ ràng để nghiên cứu sâu hơn về thiết kế heuristic bằng Generative AI dưới ràng buộc chi phí.

Tài liệu tham khảo

- [1] M. H. Ha, H. Phan, T. D. Doan, T. Dao, D. Tran, and H. T. T. Binh, “Pareto-grid-guided large language models for fast and high-quality heuristics design in multi-objective combinatorial optimization,” 2026.
- [2] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, “A fast and elitist multiobjective genetic algorithm: Nsga-ii,” *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002.
- [3] Q. Zhang and H. Li, “Moea/d: A multiobjective evolutionary algorithm based on decomposition,” *IEEE Transactions on Evolutionary Computation*, vol. 11, no. 6, pp. 712–731, 2007.
- [4] M. Laumanns, L. Thiele, and E. Zitzler, “Running time analysis of multiobjective evolutionary algorithms on pseudo-boolean functions,” vol. 8, no. 2, pp. 170–182, 2004.
- [5] Á. Fialho, L. Da Costa, M. Schoenauer, and M. Sebag, “Extreme value based adaptive operator selection,” in *Parallel Problem Solving from Nature*, 2008, pp. 175–184.
- [6] B. Romera-Paredes, M. Barekatin, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. R. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, P. Kohli, and A. Fawzi, “Mathematical discoveries from program search with large language models,” *Nature*, vol. 625, pp. 468–475, 2024.
- [7] E. Zitzler and L. Thiele, “Multiobjective evolutionary algorithms: A comparative case study and the strength pareto approach,” *IEEE Transactions on Evolutionary Computation*, vol. 3, no. 4, pp. 257–271, 1999.
- [8] C. M. Fonseca, L. Paquete, and M. López-Ibáñez, “An improved dimension-sweep algorithm for the hypervolume indicator,” in *IEEE Congress on Evolutionary Computation*, 2006, pp. 1157–1163.
- [9] J. Blank and K. Deb, “pymoo: Multi-objective optimization in python,” *IEEE Access*, vol. 8, pp. 89 497–89 509, 2020.

- [10] P. Auer, N. Cesa-Bianchi, and P. Fischer, “Finite-time analysis of the multiarmed bandit problem,” *Machine Learning*, vol. 47, pp. 235–256, 2002.
- [11] S. Bubeck and N. Cesa-Bianchi, “Regret analysis of stochastic and nonstochastic multi-armed bandit problems,” *Foundations and Trends in Machine Learning*, vol. 5, no. 1, pp. 1–122, 2012.
- [12] W. Ding, T. Qin, X.-D. Zhang, and T.-Y. Liu, “Multi-armed bandit with budget constraint and variable costs,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2013.
- [13] E. S. Page, “Continuous inspection schemes,” *Biometrika*, vol. 41, no. 1/2, pp. 100–115, 1954.